Automated Reasoning for Cryptographic Protocols

Rob Lorch

Spring 2026

1 Abstract

Security protocols are hard to prove correct. In fact, nailing down what correctness even means in the context of security protocols is itself a nontrivial question, even before attempting a proof. This difficulty has resulted in flawed "proofs" of correctness, leading to real-world security impacts. Even worse, security concerns are often eschewed in favor of financial considerations, and so proofs are often not attempted in the first place. In an ideal situation, one would like to define a security protocol, propose a security property, and have some automated tool either prove the property correct or return a concrete attack trace, all with minimal human interaction. In this document, we describe various attempts to achieve this goal, while addressing a myriad of related questions along the way.

2 Introduction

Should discuss results of various semantics more (soundness, completeness, relationships, and so on)

Introduce double colon syntax, and don't use bare tuples/pairs

Cut to 30 pages

Make sure all figs that are supposed to be in main text are in the main text

Security protocols are hard to get right. Examples abound of the discovery of inherent, design-level issues in security protocols only *after* said protocols were designed, implemented, and in use [50, 51, 58]. In this document, we argue that approaches based on *formal methods* can help address these issues by aiding in the detection of security flaws at the project design level. An often-cited example is the infamous Needham-Schroeder mutual authentication protocol [63], which we discuss in Section 6.2; its flaw was only discovered 17 years after the protocol was proposed with the help of formal tools [58]. Of course, there are also modern examples. A particularly relevant but tricky example is the 4G/5G LTE protocol stack [1, 2]. The protocols are composed of various phases and layers, and they include the composition of cryptographic reasoning with non-cryptographic reasoning (e.g., linear integer arithmetic). Formal approaches to analyzing the 4G and 5G LTE protocol stacks have shown promise: they have resulted in the discovery of numerous design-level flaws, even when making the overly optimistic simplifying assumption that all cryptographic primitives are perfectly secure [50, 51, 37]. However, the state-of-the-art techniques [50, 51] still rely on manual interaction that requires expertise in *two* separate formalisms, potentially hinting toward an area for future improvement.

Late discovery of design-level problems has huge costs, financial and otherwise—it is estimated that problems discovered earlier at the *design stage* (rather than the *implementation stage*) generally cost 5-30 times *less* to address [67]. The current industry-standard method to detect software flaws is *testing*. However, testing has limitations: (*i*) it relies on the existence of a concrete implementation, (*ii*) it can only prove the presence of bugs and never their absence, and (*iii*) designing test cases that account for the wide range of possible adversarial influence is difficult. Therefore, to facilitate early detection of design flaws, we are interested in the formal verification of *security protocols* with respect to *security properties*, even under the influence of an *adversary*, dubbed the *security protocol verification problem* (where the notions of *security protocol*, *security property*, and *adversary* will be elaborated later).

Scope. This document gives a broad overview of the formal verification of software systems with cryptographic components. However, we give special focus to verifying *cryptographic protocols* rather than *cryptographic primitives* (e.g., encryption functions, hash functions), as well as the *symbolic threat model*, which defines a set of adversary capabilities amenable to automation. Additionally, we discuss implemented tools, with a special focus on the two that dominate modern cryptographic protocol verification, Tamarin [60] and ProVerif [16].

Roadmap. We will first discuss relevant background information relating to cryptography and cryptographic protocol verification, followed by an overview of typical security properties. With this context in mind, we define the symbolic threat model and outline approaches to formalizing protocols and properties in this threat model. Then, we discuss the mechanization and automation of reasoning, along with some of the major challenges. Next, we discuss metatheoretic results in the symbolic model that help address these challenges and make automated reasoning more tractable. Finally, we conclude with a discussion of open problems in the area.

3 Security Background

We start with an overview of standard cryptography-related concepts in computer security.

3.1 Cryptography

The field of cryptography was somewhat ad hoc until recently (i.e., the 20th century). Historically, it largely centered around the study of *ciphers*, which are methods to obscure a secret message being exchanged between two parties such that only the intended recipient, and not an eavesdropper, can recover the original message. Two famous ciphers are *Caesar's cipher* [71] and *Vigenère's cipher* [71], which are widely discussed in cryptography textbooks (e.g., [71]). Historically, ciphers were invented and used until they were cracked. If nobody was yet to crack a cipher, then it was considered "secure."

In the ad hoc world, the only protection from malicious actors is the faith that they cannot crack your cipher. However, one would like to know that a cipher is *provably* secure; that is, that it is accompanied with a proof that it is impossible to "crack." To achieve this goal, more rigorous definitions of security are required. We give an overview of mathematical definitions in the setting of *symmetric-key cryptography* where a single shared key is used for both encrypting and decrypting messages; these concepts can naturally be extended to more complicated settings.

Symmetric key encryption scheme. Consider a set of messages $\mathcal{M}$, a set of ciphertexts $\mathcal{C}$, and a set of keys $\mathcal{K}$. Typically $\mathcal{M} = \mathcal{C} = \{0,1\}^*$ and $\mathcal{K} = \{0,1\}^\lambda$ for a fixed security parameter $\lambda \in \mathbb{Z}^+$. Each *plaintext* $p \in \mathcal{M}$ intuitively represents a message to be shared, each *ciphertext* $c \in \mathcal{C}$ represents an obscured plaintext, and each *key* $k \in \mathcal{K}$ can be used to help generate ciphertexts from plaintexts. A *symmetric-key encryption scheme* is a triple $(\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$ comprised of three functions $\mathsf{Gen} :: \{1\}^+ \to K$, $\mathsf{Enc} :: \mathcal{M} \times \mathcal{K} \to \mathcal{C}$, and $\mathsf{Dec} :: \mathcal{C} \times \mathcal{K} \to \mathcal{M}$, where $\mathsf{Gen}(1^\lambda)$ probabilistically samples a key of length λ from $\mathcal{K}$, and Enc and Dec denote encryption and decryption as expected. We require that for all $m \in \mathcal{M}$ and $k \in \mathcal{K}$, $\mathsf{Dec}(\mathsf{Enc}(m,k),k) = m$.

Kerckhoff's principle. *Kerckhoff's principle* states that the security of a scheme should only rely on the secrecy of keys and not of algorithms (in this case, Gen, Enc, and Dec). There are four major arguments in favor of this principle: (1) it is easier to keep a key secret than an entire encryption scheme, especially for large organizations; (2) it is easier to replace a key than an encryption scheme if information is leaked; (3) keeping one (or a small set) of encryption schemes is more practical for wide-scale deployment of applications with cryptographic algorithms; (4) standardization of cryptographic schemes lends itself to public scrutiny, including attempts at formal verification. With Kerckhoff's principle, security becomes a problem that is amenable to mathematical analysis.

Perfect secrecy. With Kerckhoff's principle in mind, how does one determine when a symmetric key encryption scheme is secure? The gold standard is *perfect secrecy*—the requirement is that for all probability distributions over $\mathcal{M}$, all $m \in \mathcal{M}$, and all $c \in \mathcal{C}$ such that $\Pr[C = c] > 0$, $\Pr[M = m \mid C = c] = \Pr[M = m]$.

Intuitively, this means that if an adversary observes a ciphertext c, it reveals no information about the value of its plaintext m.

Perfectly secret encryption schemes have an important property.

Theorem 1. *For any perfectly secret encryption scheme,* $|\mathcal{K}| \geq |\mathcal{M}|$.

However, two results follow.

Corollary 1. *If* $|k| = \lambda$ *for all* $k \in \mathcal{K}$ *and* $M = \{0,1\}^n$ *for some* n*, then for any perfectly secret encryption scheme,* $\lambda \geq n$.

Corollary 2. *Using the same key to encrypt two distinct plaintexts violates perfect secrecy.*

From Corollary 1 and Corollary 2, keys must be long (at least as long as the messages to send) and can never be reused. From these results, in practice, perfect secrecy is often too strong of a requirement. So, often times we settle for a weaker notion of security, with a common example being *IND-CPA security.*

IND-CPA security. *IND-CPA security* (**i**ndistinguishability under **c**hosen **p**laintext **a**ttack) is defined in terms of a game between a *challenger* and an *adversary*, where the scheme is deemed secure if the adversary can win the game with probability only negligibly larger than random chance. The game proceeds as follows (where $\xleftarrow{\$}$ denotes sampling from a probability distribution).

1. $k \xleftarrow{\$} \mathsf{Gen}(1^\lambda)$
2. The adversary is given knowledge of λ and oracle access to $\mathsf{Enc}(., k)$
3. The adversary chooses two messages m_1 and m_2 such that $|m_1| = |m_2|$ (with $|.|$ denoting string length)
4. $b \xleftarrow{\$} \{0,1\}$ uniformly
5. $c \xleftarrow{\$} \mathsf{Enc}(\mathsf{m_b})$
6. Adversary is given c and retains oracle access to $\mathsf{Enc}(., k)$
7. Adversary outputs b' and wins if $b' = b$

Intuitively, the adversary generates two messages m_1 and m_2, one of them is encrypted at random, and the adversary tries to guess which one. The scheme is secure if, for all probabilistic polynomial time (PPT) adversaries, Pr[Adversary wins game] $\leq .5 + \mathsf{negl}(\lambda)$, where negl is some negligible function.

3.2 Other Cryptosystem Preliminaries

Types of keys and compromises. There are different types of cryptographic keys—*long-term keys* are keys meant to be used across sessions, while *session keys* are keys only used in a single session. A *key compromise* is a scenario where the adversary has gained knowledge of some secret key, and a *principal compromise* is a scenario where the adversary controls one of the "honest" participants.

Asymmetric key crypto. Under *asymmetric key cryptography*, every principal has a *public key* and a *secret key.* The public key is publicly known, while the secret key is only known to the principal who owns it. If A (asymmetrically) encrypts m with B's public key, then it is computationally infeasible to recover m without knowledge of B's private key. Additionally, A can *digitally sign* of message m with A's private key, producing a *digital signature* which can be *verified* using A's public key. Here, it is computationally infeasible to produce a signature of a message m that will pass verification with A's public key without knowledge of A's private key. Note that a digital signature is meant to guarantee authenticity, and an asymmetric encryption is meant to guarantee secrecy.

Cryptographically secure hashes. A *cryptographically secure hash function* h has two properties:

1. It is computationally infeasible to invert (called *one-wayness*).
2. It is computationally infeasible to find distinct x and x' such that $\mathsf{h}(x) = \mathsf{h}(x')$ (called *collision resistance*).

XOR and DH exponentiation. There are other cryptographic primitives in addition to the standard encryption and decryption. *Exclusive or* is often used, as computing the (bitwise) exclusive or of a message with random bits produces an output that is indistinguishable from random bits (in fact, it satisfies perfect secrecy). Another common primitive is *Diffie–Hellman exponentiation*. In this setting, principals operate in a cyclic group G generated by an element g. Given a secret exponent x, the value g^x can be computed efficiently, but recovering x from g^x is assumed to be computationally infeasible (if not, then the group is not suitable for Diffie–Hellman exponentiation).

3.3 Cryptographic protocols

A *cryptographic protocol* is a standardized procedure followed by a group of parties communicating remotely over an insecure network channel in order to accomplish some goal, composing cryptographic primitives in an attempt to maintain a specified set of security properties. Each participant in the protocol description is called a *role*, and each concrete participant of a particular protocol instance is called a *principal* (each role can be instantiated by various principals). An *honest participant* follows the protocol description, while the *adversary* attempts to undermine the security guarantees of the protocol and can perform any action that is allowed according to a given *threat model* (i.e., description of attacker capabilities). A *session* is a single logical instance of a protocol, and a protocol *trace* is a log of events associated with the execution of a protocol, possibly containing multiple sessions.

4 Cryptographic Primitive and Protocol Verification Background

While *cryptography* is a mathematical field, one might wonder how to mechanize reasoning about cryptographic systems. To do this, approaches must address the fundamental tradeoff between the *level of automation provided by the approach* and the *degree to which the approach respects mathematical cryptography*.

4.1 General preliminaries

We use $\rightarrow^*$ to denote the reflexive-transitive closure of binary relation $\rightarrow$. We use $\mathcal{P}(S)$ to denote the power set of set S. Notation $f \circ g(x)$ denotes function composition of the form $g(f(x))$ for input x (note that we evaluate f first). A term t is *ground* if it does not contain variables. We use $\{x \mapsto M\}$ to denote the substitution of term M for variable x, and we use postfix notation to denote the application of a substitution (e.g., $t\{x \mapsto M\}$). We use notation $t[x_1, \ldots, x_n]$ to denote that term t has free variables x_1 through x_n, and $fv(t)$ to denote the free variables of term t. In discussions relating to π-calculus, we also use the prior two notations for processes (rather than terms). We use $\triangleq$ to denote "equal by definition," and we use $f[x \mapsto y]$ to denote the function that agrees with f, save for mapping x to y. Further preliminaries for more specific notation will be introduced later in the report, closer to where the notation is used.

4.2 Threat models

Approaches to verification can be classified based on the *threat model*, i.e., the characterization of the adversary's capabilities as well as the security properties to prove. Approaches can generally be split into two camps, the *computational model* and *symbolic model*. Additionally, there a middle ground called the *computationally complete symbolic attacker (CCSA) model*.

In the *computational model* [45], the semantics closely align with the mathematical formalism described in Section 3—adversaries are PPT Turing machines, functions map bit strings to bit strings, and security properties are defined in terms of games and probabilities. The computational model will be discussed further in Section 4.4.

The *symbolic threat model* [37] is characterized by the assumption that all cryptographic primitives are perfectly secure. In addition, security properties are reframed (and often weakened) to avoid reasoning about probabilities or games. All together, the symbolic model represents a generally weaker notion of

security. However, it lends itself to automation and usability, and it is still strong enough to find many issues in practice. The symbolic model will be discussed in depth in Section 6.

In the *CCSA model* [8], the set of adversarial capabilities is overapproximated (compared to the computational model), and the security property is strengthened. The approximations lend themselves to automation without weakening the notion of security as in the symbolic model. However, the approach may report spurious attacks which are not always straightforward to address. Further discussion of the CCSA model is outside the scope of this report.

4.3 Primitive vs protocol verification

The computational model can be used both to verify *implementations of cryptographic primitives* as well as *cryptographic protocols*, which can be viewed as the composition of cryptographic primitives in various ways. However, the symbolic model (and CCSA model) focus solely on the verification of cryptographic protocols, where the properties of cryptographic primitives are axiomatized. If primitives are assumed to be perfectly secure (as in the symbolic model), one might assume that their composition must also be secure (barring obvious problems, like directly exposing secret keys on a public channel). However, it is easy to compose even idealized primitives in a non-obviously insecure way.

Consider a tiny protocol with a single step, $A \to B : \langle \mathsf{sr_enc}(m, k, IV), \mathsf{h}(m) \rangle$. This denotes that role A sends to role B a message with two components, the first of which is the encryption of message m with key k, and the second of which is the hash of message m. This protocol intends to accomplish the following goals:

(0) Assume IV is fresh and k is preshared between A and B.

(1) Guarantee secrecy of message m through symmetric randomized encryption with *initialization vector* IV (essentially, a random value).

(2) Guarantee *integrity* of message m (i.e., that m was not modified in flight) with cryptographically secure hash function h.

For goal (1), idealized random encryption has stronger secrecy guarantees compared to deterministic encryption, since encrypting the same message m twice (with different IVs) will yield distinct ciphertexts. Further, for goal (2), the recipient can check that the message was not modified in flight by verifying that the hash of the plaintext (after decrypting) matches the hash sent by A. However, composing the primitives in this way is insecure—since the hash function is deterministic, the adversary can count the number of times a given message is sent. While this does not directly leak the plaintext, it is still a form of information leakage that randomized encryption is supposed to combat.

4.4 Computational Model Verification

One can encode encryption schemes, security properties, and proofs in an interactive theorem prover (ITP) such as Lean [62] or Isabelle [65]. There also exist ITPs and frameworks that are designed with computational model proofs in mind (involving cryptographic games and probability distributions) such as EasyCrypt [10] and CryptHOL [12].

Because the computational model directly captures mathematical cryptography, it can be considered a gold standard for verification.

While computational model tools can provide some level of automation, proofs are largely interactive. This report focuses on the verification of cryptographic protocols in the symbolic model, which attempts to give more automation (and hence further discussion of the computational model is out of scope).

5 Security Properties

Here, we give an intuitive overview of common security properties to prove or falsify. More precise encodings are deferred to Section 8.

5.1 General property classifications

In general, properties of systems can be classified as security properties can be partitioned into *trace properties* and *hyperproperties* [28]. *Trace properties* are defined over single traces, while *hyperproperties* are defined in terms of sets of traces. In particular, we are interested in a special class of hyper properties called *equivalence properties*, which are defined in terms of pairs of traces, and intuitively capture various notions of equivalence between two systems.

Further, trace properties can be partitioned into *safety* and *liveness* properties [28], where *safety* properties intuitively state that "something bad never happens," whereas *liveness* properties state that "something good eventually happens." The trace properties we consider in this document are all safety properties.

5.2 Standard security properties

For trace properties, we consider *secrecy*-related and *authentication*-related properties.

S_1. A protocol satisfies *weak secrecy* [17] of a term t if there is no trace where t appears in the adversary's knowledge base.

S_2. A protocol satisfies *forward secrecy* [35] if the compromise of a long-term key in the current session does not violate weak secrecy of messages shared in previous sessions. (While forward secrecy has been included for completeness, we do not discuss it further in this document.)

A_1. A protocol satisfies *non-injective agreement of B to A* [56] if, whenever principal A accepts data item d, apparently from B, then at some point in the past, B has sent data item d, apparently to A.

A_2. A protocol satisfies *injective agreement of B to A* [56] if, whenever principal A accepts data item d, apparently from B, then there exists a *unique* prior trace of B in which B sent data item d, apparently to A.

Above, S_1 and S_2 are secrecy properties, while A_1 and A_2 are authentication properties. More intuitively, property A_2 implies the absence of replay attacks—every subsequent time that A thinks that A has received a message m from B, B must have sent it again, and later acceptance of m cannot be due to replay. Note that in S_1 and S_2, secrecy is defined in terms of attacker knowledge of some term t. This is a separate notion from IND-CPA security and perfect secrecy, which both capture information leakage with probabilistic definitions.

We also consider equivalence properties. Note that these properties are more technical, so the intuition is less precise.

E_1. Two protocols P_1 and P_2 are *trace equivalent* [32] if, for every trace of P_1 under adversarial interaction, there exists a trace of P_2 (and vice versa) such that the resulting traces are indistinguishable (where *indistinguishable* is defined in terms of symbolic adversarial tests, e.g. testing equality between two attacker-computable terms).

E_2. Two protocols are *open bisimilar* [73] if they can simulate each other's observable behavior step by step under adversarial interaction, while maintaining indistinguishable adversary knowledge after each observable action.

E_3. Two protocols are *diff equivalent* [17] if, when executed in lockstep under adversarial influence, they perform corresponding steps of computation and produce adversary knowledge that remains indistinguishable at every step.

Note that E_3 is strictly stronger than E_2, which is strictly stronger than E_1.

One can use equivalences to define stronger notions of secrecy. Here, let $P(t)$ denote the protocol obtained by instantiating secret s with term t in protocol P. Then, protocol P satisfies *strong secrecy* of secret s if $P(t_1)$ is equivalent to $P(t_2)$ for all t_1 and t_2. Here, any notion of equivalence can be used, affecting the strength of the property. Intuitively, strong secrecy is stronger than weak secrecy because a protocol that leaks information about a secret without revealing it directly will falsify strong secrecy, but not weak secrecy. Of course, proving strong secrecy is desirable, and there are examples of protocols that satisfy

weak secrecy but not strong secrecy, and the violation of strong secrecy can sometimes be a major problem. Canonical examples are *padding oracle attacks* [20], where server responses to adversary-mauled ciphertexts reveal information about the underlying plaintext (without directly leaking it). In these attacks, so much information is revealed that the adversary is eventually able to reconstruct the *entire* plaintext.

One can also encode *privacy properties* using equivalences. Here, one would state that two protocols differing only in the private data being used must satisfy some notion of equivalence. The canonical examples are voting protocols which attempt to achieve various notions of vote privacy [33]—for example, in an election, an adversary should not be able to distinguish a situation where Alice voted for candidate A and Bob voted for candidate B from a situation where Alice voted for candidate B and Bob voted for candidate A.

6 Symbolic Threat Model

In this section, we define the *symbolic model*, a threat model introduced in a seminal paper by Dolev and Yao [37] (hence, it is also called the Dolev-Yao model). For now, some details are intentionally omitted, as they depend on the formalization of security protocols (discussed in Section 7). A characterizing aspect of the symbolic model is the *perfect cryptography assumption* which treats cryptographic primitives as black boxes that are perfectly secure.

Symbolic model terms. In contrast to the computational model, which treats protocol messages as bit strings, the symbolic model treats protocol messages as symbolic terms. More concretely, in the symbolic model, protocol participants (including the adversary) communicate with *order-sorted terms*, where we consider three sorts: *msg*, *fr*, and *pub*. Intuitively, *msg* is the sort for protocol messages, *fr* is the sort for fresh variables and values (e.g., nonces), and *pub* is the sort for public variables and values (e.g., principal identities). More precisely, *fr* and *pub* are incomparable and both subsorts of *msg*. Also, we consider a set of function symbols (along with their signatures) $\Sigma = \Sigma_c \cup \Sigma_d$ comprised of *constructors* (Σ_c) and *destructors* (Σ_d), as well as a set of data $D = \mathcal{N} \cup V$ consisting of atomic names $\mathcal{N}$ (representing nonces, messages, IDs, and so on) and variables V. Variables syntactically denote their sort (*e.g.*, v^{FR} has sort *fr*), and V is defined as the disjoint union of every V_s for each sort s (and respectively for $\mathcal{N}$). We denote the set of well-sorted terms (i.e., terms where function arguments have the correct sort) built from function symbols in Σ and data in D by $\mathcal{T}(\Sigma, D)$. For example, we could have $(\mathsf{s_enc}, msg \times msg \to msg) \in \Sigma_c$ and $\mathsf{s_enc}(v_1^{\text{MSG}}, v_2^{\text{FR}}) \in \mathcal{T}(\Sigma, D)$. For brevity, we may omit the MSG superscript for variables with sort *msg*. We define the *positions* of a term as usual and use $t|_p$ to denote the subterm of t at position p and $t_1[t_2]_p$ to denote substituting t_2 for the subterm at position p in t_1.

Algebraic properties of cryptographic primitives. We model cryptographic properties through a set of equations E, where each equation is comprised of two terms in $\mathcal{T}(\Sigma, V)$. In the most basic setting, we have $E = \{\mathsf{s_dec}(\mathsf{s_enc}(m, k), k) = m\}$ with $\Sigma = \{(\mathsf{s_enc}, msg \times msg \to msg), (\mathsf{s_dec}, msg \times msg \to msg)\}$. Intuitively, the equation $\mathsf{s_dec}(\mathsf{s_enc}(m, k), k) = m$ states that decrypting a ciphertext using the same key that it was encrypted with yields the underlying plaintext. We use $=_E$ to denote *equality modulo* E, where $t_1 =_E t_2$ denotes that t_1 and t_2 are related in the smallest congruence relation containing E that is closed under substitutions. For example, with this E, we get $\mathsf{enc}(\mathsf{dec}(\mathsf{enc}(d_1, k_1), k_1), k_2) =_E \mathsf{enc}(d_1, k_2)$.

Attacker capabilities. Given a set of public terms P and an insecure network N (purposefully left informal for now), we can inductively define the set of terms $\mathcal{K}$ in the knowledge base of a symbolic model attacker, as in Figure 1. Figure 1 is a collection of inference rules, where the premises (interpreted conjunctively) are on top of the bar, and the conclusion is below. PUB states that if a term t is public, then it in the adversary's knowledge; NET states that terms on the public network are in the adversary's knowledge; EQ states that once an attacker knows a term t, it knows the whole equivalence class of terms according to E; FUN states that the adversary can apply functions to terms in the adversary's knowledge; FR denotes that the adversary can sample fresh terms. The side condition on FR states that the adversary does not necessarily know *every* fresh term, but only the ones the adversary has freshly sampled. As an example, assume $\mathsf{s_enc}(m, k)$ was sent on N and that $k \in \mathcal{K}$. From this, we can apply NET to $\mathsf{s_enc}(m, k)$ and include it in $\mathcal{K}$. Then, because k and $\mathsf{s_enc}(m, k)$ are both in $\mathcal{K}$, we can add $\mathsf{s_dec}(\mathsf{s_enc}(m, k), k)$ to $\mathcal{K}$ (using FUN). Finally, with EQ, since $\mathsf{s_dec}(\mathsf{s_enc}(m, k), k) =_E m$, we can add m to $\mathcal{K}$. Note that EQ implicitly encodes the perfect cryptography

$$\frac{t \in P}{t \in \mathcal{K}}\ \textsc{Pub} \qquad \frac{t \text{ was sent on } N}{t \in \mathcal{K}}\ \textsc{Net} \qquad \frac{t \in \mathcal{K} \quad t =_E t'}{t' \in \mathcal{K}}\ \textsc{Eq} \qquad \frac{t_1, \ldots, t_n \in \mathcal{K} \quad (f, n) \in \Sigma}{f(t_1, \ldots, t_n) \in \mathcal{K}}\ \textsc{Fun}$$

$$\frac{t^{\textsc{Fr}} \in D}{t^{\textsc{Fr}} \in \mathcal{K}}\ \textsc{Fr} \text{ (when } t \text{ is fresh)}$$

Expressions above each bar are premises, and statements below are conclusions. $\mathcal{K}$ denotes the set of terms in the adversary's knowledge, P denotes public terms, and D denotes data items (e.g., nonces), and Σ denotes the function signature.

Figure 1: Inference rules defining symbolic adversary knowledge $\mathcal{K}$

assumption—the only way to retrieve the encrypted m is to decrypt with the same key used to encrypt, and there is no other equation in E to retrieve m (and there is no adversary inference rule that directly destructs terms). Note that $\mathcal{K}$ is defined as the *smallest* set satisfying these rules. Although it is not reflected in the knowledge rules, the adversary can additionally send any term from $\mathcal{K}$ along N.

Security properties. The Dolev-Yao model does not directly inherit security properties from the computational model, as we want to avoid reasoning about probabilities (as one does with perfect secrecy or IND-CPA secrecy). For example, the symbolic model uses a weaker notion of secrecy, aptly called *weak secrecy.* A protocol satisfies weak secrecy if it is never the case that any term marked secret is in $\mathcal{K}$ (where "marked secret" will be elaborated in Sections 8.2 and 8.3). We gave an informal overview of security properties in Section 5; more detailed encodings will follow in Section 8.

Relationship to computational model. It is tempting to say that the adversarial capabilities in the symbolic model are a strict underapproximation of adversarial capabilities in the computational model (i.e., that every symbolic model attack has a corresponding attack in the computational model, and the symbolic attacker is strictly weaker than the computational attacker). From this, it would also follow that every proof of security in the computational model would also hold with respect to a symbolic attacker. *However, this view is not precise.* In one axis, the symbolic model adversary is *stronger* than the computational model adversary—there is no notion of a complexity bound in the symbolic world, while there is in the computational world (the adversary must compute in PPT); there is no guarantee that a derivation from the rules in Figure 1 is computationally realizable in PPT. For example, a Dolev-Yao adversary could invoke the Eq rule with the equation $\mathsf{invert}(\mathsf{h}(x)) = x$ (if it is present in E). More generally, stating a precise connection between the symbolic and computational worlds is nontrivial because they live in different semantic worlds. While outside the scope of this document, the most common bridge is in *computational soundness* proofs that demonstrate situations in which symbolic model security soundly implies computational model security [4].

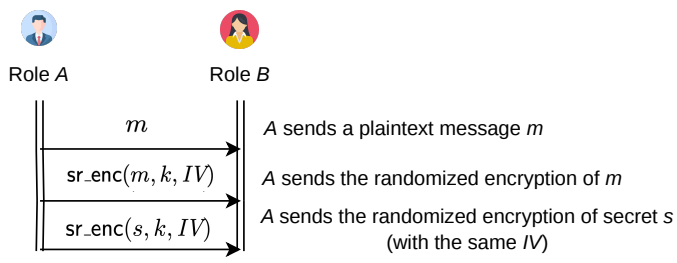
6.1 Weaknesses of the symbolic model

Note that the only way for a Dolev-Yao adversary to destruct terms is through Eq, which (for example) lets the attacker derive m from $\mathsf{dec}(\mathsf{enc}(m, k), k)$. This exhibits the perfect cryptography assumption—the *only way* to recover the plaintext from a ciphertext is to have knowledge of the key. Because of this, the symbolic model is not *computationally proof sound* in the sense that a protocol that is insecure in the computational model may have no attack (be "secure") in the symbolic model. An example protocol is given in Figure 2, where we want to ensure weak secrecy of s (i.e., the attacker cannot directly compute it).

Symmetric random encryption and decryption is modeled by

$$\Sigma = \{(\mathsf{sr_enc}, msg \times msg \times msg \to msg), (\mathsf{sr_dec}, msg \times msg \times msg \to msg)\} \text{ and}$$
$$E = \{\mathsf{sr_dec}(\mathsf{sr_enc}(m, k, IV), k) = m\}.$$

The protocol is computationally insecure, as an initialization vector should never be reused; in this protocol, however, IV is used in both the second and third message.



sr_enc denotes symmetric, randomized encryption. IV denotes an *initialization vector*, providing a source of randomness for the encryption.

Figure 2: Protocol with IV reuse

Justification	Computation	Adversary knowledge
definition of sr_enc	$\mathsf{sr_enc}(m,k,IV) \oplus m$ $= \mathsf{keystream}(k, IV)$	$m, \mathsf{sr_enc}(m,k,IV), \mathsf{sr_enc}(s,k,IV)$
definition of sr_enc associativity of $\oplus$ cancellation of $\oplus$	$\mathsf{sr_enc}(s,k,IV) \oplus \mathsf{keystream}(k,IV)$ $= (s \oplus \mathsf{keystream}(k,IV)) \oplus \mathsf{keystream}(k,IV)$ $= s \oplus (\mathsf{keystream}(k,IV) \oplus \mathsf{keystream}(k,IV))$ $= s$	$m, \mathsf{sr_enc}(m,k,IV), \mathsf{sr_enc}(s,k,IV), \mathsf{keystream}(k,IV)$

Blue terms denote terms in the adversary's knowledge at the current step.
Red terms denote terms computed by the adversary in this step.
Black terms denote intermediate terms illustrating the derivation.

Figure 3: IV reuse attack computing secret s

More concretely, suppose sr_enc is defined as follows: $\mathsf{sr_enc}(m,k,IV) = m \oplus \mathsf{keystream}(k,IV)$, where $\oplus$ denotes exclusive or and keystream builds a stream of pseudo-random bits from k and IV (the details of keystream are not necessary for this discussion). In this protocol, the adversary can compute s, revealing the secret! The derivation is outlined in Figure 3. In the first computation, notice that $\mathsf{sr_enc}(m,k,IV) = m \oplus \mathsf{keystream}(k,IV)$ (by definition of sr_enc), which implies $\mathsf{keystream}(k,IV) = \mathsf{sr_enc}(m,k,IV) \oplus m$. Because $\mathsf{sr_enc}(m,k,IV)$ and m are both sent along the channel, the adversary can compute $\mathsf{keystream}(k,IV)$. In the second step, notice that $\mathsf{sr_enc}(s,k,IV) = s \oplus \mathsf{keystream}(k,IV)$ (by definition of sr_enc), which implies $s = \mathsf{sr_enc}(s,k,IV) \oplus \mathsf{keystream}(k,IV)$. Because IV was erroneously reused, the attacker still knows the keystream and (along with knowledge of $\mathsf{sr_enc}(s,k,IV)$) can directly compute s.

However, in the symbolic model, there is no attack, as the attacker needs knowledge of k to destruct $\mathsf{sr_enc}(s,k,IV)$ and retrieve s. However, k is not derivable by the adversary from the rules in Figure 1.

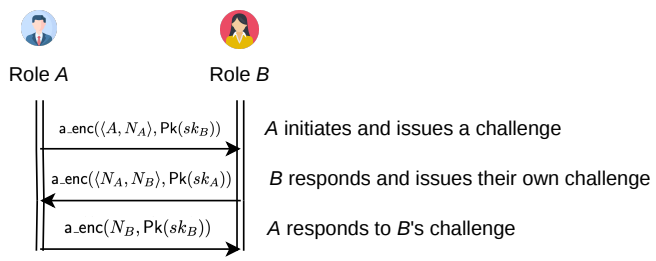
6.2 Symbolic model attacks

Despite its limitations, the symbolic model can still find interesting attacks in practice. A famous example is the attack found on the Needham-Schroeder public key authentication protocol [58], which was discovered using cryptographic verification tools 17 years after the protocol was originally published [58]. It uses asymmetric key cryptography, pairing, and projection, modeled by

$$\Sigma = \{(\mathsf{a_enc}, msg \times msg \to msg), (\mathsf{a_dec}, msg \times msg \to msg), (\mathsf{pk}, msg \to msg),$$
$$(\langle .,. \rangle, msg \to msg), (\pi_1, msg \to msg), (\pi_2, msg \to msg)\} \text{ and}$$
$$E = \{\mathsf{a_dec}(\mathsf{a_enc}(m, \mathsf{pk}(sk)), sk) = m, \pi_1(\langle t_1, t_2 \rangle) = t_1, \pi_2(\langle t_1, t_2 \rangle) = t_2\}.$$

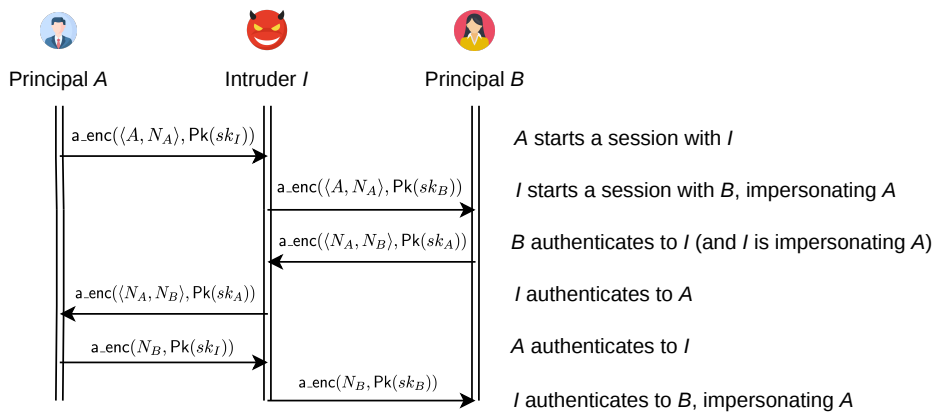
The protocol is presented in Figure 4.

The protocol attempts to mutually authenticate A and B. Intuitively, from A's perspective, B is authenticated because B was able to decrypt the first message with B's secret key and return N_A. Analogously, from B's perspective, A is authenticated because A was able to decrypt the second message with A's secret



A and B denote protocol roles. $\mathsf{a_enc}(m, k)$ denotes asymmetric encryption with message m and key k, and $\mathsf{Pk}(sk)$ denotes the public key derived from secret key sk.

Figure 4: The original Needham–Schroeder public-key protocol



I denotes the intruder. A and B denote principals. $\mathsf{a_enc}(m, k)$ denotes asymmetric encryption with message m and key k, and $\mathsf{Pk}(sk)$ denotes the public key derived from secret key sk.

Figure 5: Attack on the original Needham–Schroeder public-key protocol (in Figure 4)

key and return N_B. However, the protocol is symbolically insecure. Consider the attack trace with intruder I presented in Figure 5. (By inspection, one can verify that all messages sent by I are deducible from the rules in Figure 1.)

Above, A knows that they are communicating with I, but B believes that they are communicating with A (when B is actually communicating with I). How was I able to impersonate A to B? In message (2), I can simply copy A's challenge because message (1) was encrypted with I's public key. Then I can respond to B's challenge in step (3) by forwarding it to A in step (4).

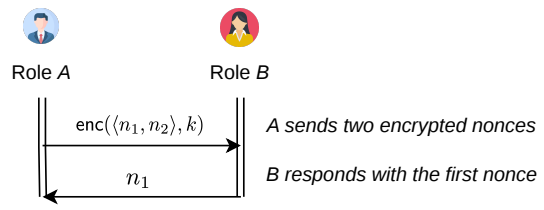
Conceptually, the protocol is insecure because B's challenge is not tied to B's identity—the fix is to substitute $\langle N_A, N_B, B\rangle$ for $\langle N_A, N_B\rangle$ in step (b) (in Figure 5, step (3)). Then if I forwards in step (4), it will become clear to A that there is an identity mismatch between I and B.

The Needham-Schroeder protocol is a three-step protocol, and yet it requires care and understanding to avoid symbolic model attacks. Thus, we can argue that for larger and more complicated protocols, symbolic model verification can eliminate relevant classes of attacks, despite its limitations.

7 Formalizing Protocols in the Symbolic Model

We aim to mechanize reasoning about cryptographic protocols. To do this, we need to rigorously formalize properties, protocols, and adversarial influence. Notice that describing protocols as sequences of rules $A \rightarrow$

$B : M$ (as we have done so far) is insufficient for precise reasoning. For example, the encoding leaves the notion of *public knowledge* implicit (e.g., identities and public keys are initially public). Further, it does not describe the *inputs* and *internal steps* of honest participants. Consider a small protocol as in Figure 6.



Standard security protocol notation leaves some paths unspecified—what happens if B cannot retrieve n_1?

Figure 6: Ambiguous toy protocol

It is left implicit that in the second step, B should decrypt the incoming message, retrieve the first element of the pair (n_1), and send it back to A. But what happens if decryption or projection fails? It is not specified.

There are four desirable properties when choosing a formalism to encode protocols and protocol traces in the symbolic model.

1. The formalism must be able to model *message passing* of symbolic terms between parties, taking into account algebraic properties for cryptographic primitives (e.g., equational theory E in Section 6).
2. The formalism must be able to encode a *nondeterministic adversary* tampering with communications following a predefined set of capabilities.
3. The formalism must be able to model the interweaving of multiple distinct sessions, both in parallel and in sequence, allowing the adversary to mix messages of various sessions (say, learning message m from session 1 and injecting it along the network channel in session 2).
4. The formalism would ideally be *executable* in the sense that it should be straightforward to generate protocol traces given a protocol description.

From these requirements, two approaches have historically dominated the formalization of security protocols in the symbolic model: those based on *multiset rewriting*, and those based on *applied π-calculus*.

7.1 Multiset rewriting-based approaches

The multiset-rewriting approach to cryptographic protocol formalization was pioneered by Durgin and Mitchell in 1999 [40]. It is an important formalism for modern automated reasoning techniques in the symbolic model, as it lays the theoretical foundations for Tamarin [60], a popular symbolic model verification tool. In this approach, a protocol is a set of multiset rewriting rules, where each rewrite rule can be viewed as a state transition in the protocol.

Formalizing protocols. We superscript standard set operations with $^{\#}$ to denote multiset operations (e.g., $\cup^{\#}$ denotes multiset union). To formalize protocols, we consider another signature Σ_{FACT} of *fact symbols*, where *facts* are members of the set $\{F(t_1, \ldots, t_n) \mid F \in \Sigma_{\text{FACT}} \ \wedge \ t_1, \ldots, t_n \in T(\Sigma, V \cup \mathcal{N})\}$. A protocol is represented as a set of rewrite rules R, where each rule in R is a triple (l, a, r), where l, a, and r are each finite multisets of facts. Sometimes, we use the alternative notation $l \xrightarrow{a} r$ to denote this triple: l and r respectively represent the left-hand side and right-hand side of the rewrite rule (containing *state facts*), while a represents rule labels (containing *action facts*). More formally, for a protocol R, the set of action facts $\mathcal{A}$ is defined as $\{af \mid \exists a.(_, a, _) \in R \wedge af \in^{\#} a\}$, and the set of state facts $\mathcal{F}$ is defined as $\{sf \mid \exists l, r.(l, _, _) \in R \wedge (_, _, r) \in R \wedge (sf \in^{\#} l \vee sf \in^{\#} r)\}$. For a rewrite rule $l \xrightarrow{a} r$, state facts in l are called *premises* and state facts in r are called *conclusions*.

$$\mathsf{Adv} = \left\{ \begin{array}{rcl} \{\mathsf{Out}(x)\} & \xrightarrow{\{\}} & \{\mathsf{K}(x)\} \\ \{\mathsf{Fr}(x^{\text{FR}})\} & \xrightarrow{\{\}} & \{\mathsf{K}(x^{\text{FR}})\} \\ \{\} & \xrightarrow{\{\}} & \{\mathsf{K}(x^{\text{PUB}})\} \\ \{!\mathsf{K}(x)\} & \xrightarrow{\{\mathsf{K}(x)\}} & \{\mathsf{In}(x)\} \end{array} \right\} \cup \left\{ \begin{array}{c} \{\mathsf{K}(t_1), \ldots, \mathsf{K}(t_n)\} \xrightarrow{\{\}} \{\mathsf{K}(f(t_1, \ldots, t_n))\} \\ \text{for all } (f, n) \in \Sigma \end{array} \right\}$$

Each triple $l \xrightarrow{a} r$ denotes a multiset rewriting rule from l to r with labels a. Out (resp., In) denotes output (resp., input) along the network channel, Fr denotes fresh terms, and K denotes intruder knowledge.

Figure 7: Adversary rules for multiset rewriting formalization

Formalizing the adversary. To formalize adversarial behavior, we assume the existence of four special fact symbols In, Out, Fr, and K. In and Out are included in the protocol's rules to model taking input and sending output to the insecure network channel. Fr represents fresh variables (where freshness is later enforced in the operational semantics), and K represents adversary knowledge and should not be present in protocol rules. Adversarial behavior can be captured by a fixed set of rewrite rules Adv that roughly mirror the rules in Figure 1. Adv is defined in Figure 7: the first three rules on the left correspond to Net, Fr, and Pub, respectively; the last rule on the left corresponds to the adversary's capability to inject messages on the network; and the rules on the right correspond to Fun.

In the last rule on the left in Figure 7, the bang (!) is shorthand to denote that the fact should be repeated on the right-hand side of the rule. Facts with bangs are called *persistent facts*, as they represent persistent state. For clarity, a fact should never be persistent in one rule and *linear* (i.e., not persistent) in another rule.

Protocol traces. A protocol trace is a sequence of states, where each successive pair of states follows from one of the rules in $R \cup \mathsf{Adv} \cup \{\mathsf{Fresh}\}$, where Fresh denotes the built-in rule for fresh variables $\{\} \xrightarrow{\{\}} \{\mathsf{Fr}(x)\}$. More formally, we use σ to denote a substitution from variables to terms, and we use postfix notation $t\sigma$ to denote the application of the homomorphic extension of σ to term t. We abuse notation and also use postfix notation $m\sigma$ to denote the application of (the homomorphic extension of) σ to all arguments of all facts in multiset m. We further abuse notation and write $\sigma_1 \circ \sigma_2$ to denote the composition of substitutions, where we apply the homomorphic extension of σ_2 to the output σ_1 (which itself may also be interpreted as a homomorphic extension depending on context). Let $\mathsf{ginsts}(l \xrightarrow{a} r)$ denote the ground instances of a rewrite rule $l \xrightarrow{a} r$, i.e.,

$$\mathsf{ginsts}(l \xrightarrow{a} r) = \{l_g \xrightarrow{a_g} r_g \mid \exists \sigma.\ l\sigma = l_g \wedge a\sigma = a_g \wedge r\sigma = r_g \wedge \sigma \text{ only maps to ground terms}\}.$$

Further, we use $\mathsf{ginsts}_E(l \xrightarrow{a} r)$ to denote the ground instances of $l \xrightarrow{a} r$ modulo E, i.e.,

$$\mathsf{ginsts}_E(l \xrightarrow{a} r) = \{l' \xrightarrow{a'} r' \mid \exists l_g \xrightarrow{a_g} r_g \in \mathsf{ginsts}(l \xrightarrow{a} r) \wedge l' =_E l_g \wedge a' =_E a_g \wedge r' =_E r_g\}.$$

Finally, we lift $\mathsf{ginsts}_E(.)$ from single rewrite rules to sets of rewrite rules as expected. From this, we can define a small-step labeled transition relation for a set of rewrite rules R, denoted $\xrightarrow{a}_R$, with the following inference rule. Here, assume S denotes the current state (which is a multiset of facts) and $\mathcal{S}$ denotes the set of all states, and hence $\xrightarrow{a}_R \subseteq \mathcal{P}(\mathcal{S}) \times \mathcal{P}(\mathcal{S})$.

$$\frac{l \xrightarrow{a} r \in \mathsf{ginsts}_E(R) \quad l \subset^{\#} S}{S \xrightarrow{a}_R (S \setminus^{\#} l) \cup^{\#} r}$$

Intuitively, the state S evolves at each step through the application of some rewrite rule in R that consumes state facts on the left-hand side of the rule and produces state facts on the right-hand side of the rule. Finally, a *trace* of a protocol R is a sequence of states $S_0, \ldots, S_n$ such that $S_0 = \emptyset$ and $S_i \xrightarrow{a}_R S_{i+1}$ for all i

$$
\begin{aligned}
\mathsf{Register} &\triangleq \{\mathsf{Fr}(sk^{\text{Fr}})\} \xrightarrow{\{\}} \\
&\quad \{\mathsf{Ltk}(A^{\text{Pub}}, sk^{\text{Fr}}), \mathsf{Pk}(A^{\text{Pub}}, \mathsf{pk}(sk^{\text{Fr}})), \mathsf{Out}(\langle A^{\text{Pub}}, \mathsf{pk}(sk^{\text{Fr}})\rangle)\}, \\
\mathsf{Reveal} &\triangleq \{!\mathsf{Ltk}(A^{\text{Pub}}, sk^{\text{Fr}})\} \xrightarrow{\{\}} \\
&\quad \{\mathsf{Out}(sk^{\text{Fr}})\rangle)\}, \\
A_1 &\triangleq \{\mathsf{Fr}(n_A^{\text{Fr}}), !\mathsf{Pk}(A^{\text{Pub}}, pk_A^{\text{Pub}}), !\mathsf{Pk}(B^{\text{Pub}}, pk_B^{\text{Pub}})\} \xrightarrow{\{\}} \\
&\quad \{\mathsf{StA1}(A^{\text{Pub}}, B^{\text{Pub}}, n_A^{\text{Fr}}), \mathsf{Out}(\mathsf{a_enc}(\langle A^{\text{Pub}}, n_A^{\text{Fr}}\rangle, pk_B^{\text{Pub}}))\}, \\
B_1 &\triangleq \{\mathsf{In}(c^{\text{Pub}}), !\mathsf{Ltk}(B^{\text{Pub}}, sk_B^{\text{Fr}}), !\mathsf{Pk}(A^{\text{Pub}}, pk_A^{\text{Pub}}), \mathsf{Fr}(n_B^{\text{Fr}})\} \xrightarrow{\{\}} \\
&\quad \{\mathsf{StB1}\big(B^{\text{Pub}}, \pi_1(\mathsf{a_dec}(c^{\text{Pub}}, sk_B^{\text{Fr}})), \pi_2(\mathsf{a_dec}(c^{\text{Pub}}, sk_B^{\text{Fr}})), n_B^{\text{Fr}}\big), \\
&\qquad \mathsf{Out}\big(\mathsf{a_enc}(\langle \pi_2(\mathsf{a_dec}(c^{\text{Pub}}, sk_B^{\text{Fr}})), n_B^{\text{Fr}}\rangle, pk_A^{\text{Pub}})\big)\}, \\
A_2 &\triangleq \{\mathsf{In}(c^{\text{Pub}}), !\mathsf{Ltk}(A^{\text{Pub}}, sk_A^{\text{Fr}}), !\mathsf{Pk}(B^{\text{Pub}}, pk_B^{\text{Pub}}), \mathsf{StA1}(A^{\text{Pub}}, B^{\text{Pub}}, n_A^{\text{Fr}})\} \xrightarrow{\{\}} \\
&\quad \{\mathsf{StA2}\big(A^{\text{Pub}}, B^{\text{Pub}}, n_A^{\text{Fr}}, \pi_2(\mathsf{a_dec}(c^{\text{Pub}}, sk_A^{\text{Fr}}))\big), \\
&\qquad \mathsf{Out}\big(\mathsf{a_enc}(\pi_2(\mathsf{a_dec}(c^{\text{Pub}}, sk_A^{\text{Fr}})), pk_B^{\text{Pub}})\big)\}, \text{ and} \\
B_2 &\triangleq \{\mathsf{In}(c^{\text{Pub}}), !\mathsf{Ltk}(B^{\text{Pub}}, sk_B^{\text{Fr}}), \mathsf{StB1}(B^{\text{Pub}}, A^{\text{Pub}}, n_A^{\text{Pub}}, n_B^{\text{Fr}})\} \xrightarrow{\{\}} \\
&\quad \{\mathsf{Commit}(B^{\text{Pub}}, A^{\text{Pub}}, n_A^{\text{Pub}}, n_B^{\text{Fr}})\}.
\end{aligned}
$$

Each triple $l \xrightarrow{a} r$ denotes a multiset rewriting rule from l to r with labels a, and facts with "!" denote persistent facts. For example, $\mathsf{Register}$ is the name of a rewrite rule that consumes $\mathsf{Fr}(sk^{\text{Fr}})$ from the state (i.e., a fresh secret key) and introduces to the state a longterm key (stored in $\mathsf{Ltk}(.,)$), a public key (stored in $\mathsf{Pk}(.,.)$), and network channel output (stored in $\mathsf{Out}(.)$).

Figure 8: Rewrite rules formalizing Figure 4

(for some $l \xrightarrow{a} r \in \mathsf{ginsts}_E(R)$), with the additional well-formedness requirement that the rule $\{\} \xrightarrow{\{\}} \{\mathsf{Fr}(x)\}$ is used at most once for a given instantiation of x.

Example protocol formalization. We will formalize the (insecure) Needham Schroeder protocol from Figure 4, as well as the attack trace. Consider

$$
\begin{gathered}
\Sigma = \{(\mathsf{a_enc}, msg \times msg \to msg), (\mathsf{a_dec}, msg \times msg \to msg), (\mathsf{pk}, msg \to msg), \\
(\langle .,. \rangle, msg \times msg \to msg), (\pi_1, msg \to msg), (\pi_2, msg \to msg)\}, \\
E = \{\mathsf{a_dec}(\mathsf{a_enc}(m, \mathsf{pk}(k)), k) = k, \pi_1(\langle t_1, t_2\rangle) = t_1, \pi_2(\langle t_1, t_2\rangle) = t_2\}, \text{ and} \\
\Sigma_{\text{Fact}} = \{(\mathsf{In}, msg), (\mathsf{Out}, msg), (\mathsf{Fr}, fr), (\mathsf{K}, msg), (\mathsf{Pk}, msg \times msg), (\mathsf{Ltk}, msg \times msg), \\
(\mathsf{StA1}, msg \times msg \times msg), (\mathsf{StB1}, msg \times msg \times msg \times msg), (\mathsf{StA2}, msg \times msg \times msg \times msg)\}.
\end{gathered}
$$

The protocol is encoded as a set of rewrite rules $R = \{\mathsf{Register}, A_1, B_1, A_2, B_2\}$, where the rewrite rules are defined as in Figure 7.1.

The first rule, $\mathsf{Register}$, represents the registration of an asymmetric key pair to identity A^{Pub}. Note that A^{Pub}, along with the other identifiers in the rule descriptions, are *variables* and can be instantiated with different values in different rule instances. Hence, the protocol description can capture an unbounded number of sessions and principals. The Ltk (short for "long-term key") stores A^{Pub}'s (fresh) secret key sk^{Fr}, while Pk stores the public key derived from sk^{Fr}. Finally, the identity/public key mapping are broadcast along the network channel, as they are public information that should be known by the adversary for realistic modeling.

The second rule, Reveal, leaks a secret key on the network channel. This models *principal compromise*, a situation where one or more of participants is corrupted and controlled by the adversary. A rule for revealing keys tests the *compromise resilience* of a protocol—one can test the security of a protocol with limitations imposed on the Reveal rule (unrestricted use trivially compromises security). In the attack on the Needham-Schroeder protocol, Reveal is needed to give intruder I knowledge of his own secret key; however, the attack does not rely on revealing the secret key of honest role A or B.

Rules A_1, B_1, and A_2 closely mirror the rules in Figure 4; however, they explicitly reference state to retrieve the corresponding keys. Additionally, state fact symbols StA1, StB1, and StA2 are used to ensure that protocol traces follow *ordering constraints* (i.e., that protocol rule i is executed before protocol rule $i+1$ in a given session), as well as to store information about the current session.

Rule B_2 is used to explicitly denote cases where B believes that B has successfully completed a protocol session with A.

Example protocol trace formalization. We can now formalize the attack trace in Figure 5 as a trace of $R \cup \mathsf{Adv} \cup \{\mathsf{Fresh}\}$ (see Figure 9). The full trace is far too verbose to be useful; we outline the first few steps representing key generation. At each step, we outline the rule used and the updates to state, which (for clarity in presentation) is partitioned between the linear state L and persistent state P (each a multiset of ground facts).

Rule application	**State**
NA	$S_0 = \emptyset$
$\{\} \xrightarrow{\{\}} \{\mathsf{Fr}(sk_A)\}$	$\{\mathsf{Fr}(sk_A)\}$
$\{\mathsf{Fr}(sk_A)\} \xrightarrow{\{\}}$ $\{\mathsf{Ltk}(A, sk_A), \mathsf{Pk}(A, \mathsf{pk}(sk_A)), \mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle)\}$	$\{\mathsf{Ltk}(A, sk_A), \mathsf{Pk}(A, \mathsf{pk}(sk_A)), \mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle)\}$
$\{\} \xrightarrow{\{\}} \{\mathsf{Fr}(sk_I)\}$	$\{\mathsf{Ltk}(A, sk_A), \mathsf{Pk}(A, \mathsf{pk}(sk_A)), \mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Fr}(sk_I)\}$
$\{\mathsf{Fr}(sk_I)\} \xrightarrow{\{\}}$ $\{\mathsf{Ltk}(I, sk_I), \mathsf{Pk}(I, \mathsf{pk}(sk_I)), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle)\}$	$\{\mathsf{Ltk}(A, sk_A), \mathsf{Pk}(A, \mathsf{pk}(sk_A)), \mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle),$ $\mathsf{Ltk}(I, sk_I), \mathsf{Pk}(I, \mathsf{pk}(sk_I)), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle)\}$
...	...

The left column outlines a sequence of ground instantiations of multiset rewriting rules denoting a trace prefix, and the right column outlines the corresponding updates to the state. Each rewrite rule consumes facts from the state (on the left-hand of the rule) and introduces new facts to the state (on the right-hand side of the rule).

Figure 9: Needham–Schroeder public-key protocol attack as multiset rewriting protocol trace prefix

For the full trace, see Figures 13 and 14 in the appendix.

Security property formalization. In this setting, given a protocol R, weak secrecy of a term t is phrased in terms of the existence of some protocol trace with some step $S_i \xrightarrow{\mathsf{K}(t)} S_{i+1}$ (where t is in the adversary's knowledge base, captured by action fact $\mathsf{K}(t)$). That is, a protocol satisfies weak secrecy of t if there is no trace with such a step.

More concretely, we consider security properties φ in a variant of first-order logic with equality that replaces predicates $p(t_1, \ldots, t_n)$ with *actions* of the form $f(t_1, \ldots, t_n)@i$ (where $f(t_1, \ldots, t_n)$ is some action fact, and i is a temporal variable). In this logic, we define satisfaction relation $tr, \theta \models_E \varphi$ which means that trace tr with valuation θ satisfies formula φ, modulo equational theory E. More details of this property specification logic will be described in Section 8.

As an intuitive example, one can specify a secrecy property as $\forall x.\ \forall i.\ \mathsf{Secret}(x)@i \Rightarrow \neg(\exists j.\mathsf{K}(x)@j)$. Intuitively, if x is marked as secret by some action fact in the protocol description (at any time i), then x does not appear in the adversary's knowledge base (at any time j).

7.2 π-calculus-based approaches

Symbolic model protocol analysis approaches based on π-calculus were pioneered by Abadi and Fournet in [3], and lay the theoretical foundations for the popular automated reasoning tool ProVerif [16]. Additionally, many theoretical results in symbolic model verification (discussed in Section 10) are expressed in terms of the π-calculus. Intuitively, the π-calculus is a minimal language that captures the concurrent execution of a collection of processes with message passing of atomic names, also supporting the dynamic creation of new communication channels.

When reasoning about cryptographic protocols, we use an extension of the π-calculus, called the *applied π-calculus*, where processes exchange symbolic terms in addition to atomic channel names. These terms represent cryptographic messages, and we can use the same set of terms $\mathcal{T}(\Sigma, D)$ as in the multiset rewriting formalization, except that D also contains channel names.

π-calculus preliminaries. An evaluation context C is a process with a hole (denoted $\square$) in it, and $C[P]$ denotes filling the hole in C with process P. For example, with context $C \triangleq \mathsf{if}\ M = N\ \mathsf{then}\ \square\ \mathsf{else}\ P$, then $C[0]$ denotes $\mathsf{if}\ M = N\ \mathsf{then}\ 0\ \mathsf{else}\ P$.

Syntax. Below is the grammar for *plain processes*, where c ranges over channel names and variables, n over names, x over variables, and M and N over terms.

$P ::=$	plain processes
0	null process
$P_1 \mid P_2$	parallel composition
$!P$	replication
$\textbf{new}\ n.\ P$	name restriction
$\mathsf{if}\ M = N\ \mathsf{then}\ P_1\ \mathsf{else}\ P_2$	conditional
$c(x).P$	message input on channel c
$\bar{c}\langle N \rangle$	message output on channel c

It is close to standard π-calculus, except for sending terms (rather than just atomic names) as output, as well as conditionals with equality tests on terms. The operation $\textbf{new}\ n.\ P$ creates a fresh name and binds it to n, then proceeds with P.

Additionally, we introduce substitutions of the form $\{x \mapsto M\}$ directly into the core language, where x is a variable and M is a term. To distinguish object-level substitutions from meta-level substitutions, we call object-level substitutions *active substitutions*. The set of a process's active substitutions can be viewed as the *adversary knowledge*. Intuitively, the adversary process can sample fresh terms and use terms in scope (i.e., in its knowledge), but terms outside of the set of active substitutions (like unleaked secret keys) are not guessable.

$Q ::=$	extended processes
P	plain process
$Q_1 \mid Q_2$	parallel composition
$\textbf{new}\ n.\ Q$	name restriction
$\textbf{new}\ v.\ Q$	variable restriction
$\{x \mapsto M\}$	active substitution

Structural equivalence. The first step to defining the operational semantics of the applied π-calculus is to define *structural equivalence* $\equiv$, a binary relation that intuitively captures the equivalence of two processes without taking forward steps of computation (i.e., the structural equivalence rules may be applied in both directions).

$\equiv$ is defined as the smallest relation closed under α-renaming on names and variables, and by application of evaluation contexts, containing the following pairs. Below, $fn(.)$ returns the free names of its input.

(1)	A	$\equiv$	$A \mid 0$	null process
(2)	$A \mid (B \mid C)$	$\equiv$	$(A \mid B) \mid C$	associativity of parallel composition
(3)	$A \mid B$	$\equiv$	$B \mid A$	commutativity of parallel composition
(4)	$!P$	$\equiv$	$P \mid !P$	replication
(5)	$\textbf{new}\ n.\ 0$	$\equiv$	0	restriction over the null process
(6)	$\textbf{new}\ n_1.\ \textbf{new}\ n_2.\ Q$	$\equiv$	$\textbf{new}\ n_2.\ \textbf{new}\ n_1.\ Q$	commutativity of restrictions
(7)	$Q_1 \mid \textbf{new}\ n.\ Q_2$	$\equiv$	$\textbf{new}\ n.\ (Q_1 \mid Q_2)$ (when $n \notin (fv(Q_1) \cup fn(Q_1))$)	scope extension of restriction
(8)	$\textbf{new}\ x.\ \{x \mapsto M\}$	$\equiv$	0	restriction of active substitution
(9)	$\{x \mapsto M\} \mid A$	$\equiv$	$\{x \mapsto M\} \mid A\{x \mapsto M\}$	application of active substitution
(10)	$\{x \mapsto M\}$	$\equiv$	$\{x \mapsto N\}$ (when $M =_E N$)	equational reasoning on substitutions

Cases (1) through (7) are standard in π-calculus. Case (8) intuitively captures the idea that an active substitution for a fresh variable x, with no other processes in scope to use x, cannot directly perform computation. Case (9) denotes the application of an active substitution. Case (10) captures the interchangeability of M and N when $M =_E N$ (note that we can use the same notion of $=_E$ from Section 7.1).

Operational semantics. We define a small-step relation $\rightarrow$, which is the smallest relation closed under structural equivalence and evaluation contexts such that the following hold.

(1)	$\bar{c}\langle x\rangle.\ Q_1 \mid c(x).\ Q_2$	$\rightarrow$	$Q_1 \mid Q_2$	message passing
(2)	$\mathsf{if}\ M = M\ \mathsf{then}\ Q_1\ \mathsf{else}\ Q_2$	$\rightarrow$	Q_1	then branch
(3)	$\mathsf{if}\ M = N\ \mathsf{then}\ Q_1\ \mathsf{else}\ Q_2$	$\rightarrow$	Q_2 (when $M \neq_E N$)	else branch

Above, rule (1) represents sharing a message across a shared channel c. It might seem strange to require a variable (rather than any term) to be sent along c. However, this restriction is without loss of generality because a process can send term M by introducing a fresh variable x and active substitution $\{x \mapsto M\}$ using the structural equivalence rules above.

Example protocol. We encode the original Needham-Schroeder protocol from Figure 4 in the applied π-calculus. For clarity, we introduce two aspects of syntactic sugar. First, we introduce *let expressions* of the form $\mathsf{let}\ x = e_1\ \mathsf{in}\ e_2$. Second, we introduce *abstractions* of the form $\mathsf{Name}(x_1, \ldots, x_n) \triangleq P[x_1, \ldots, x_n]$, which are functions from terms to processes and can be called with the usual syntax $\mathsf{Name}(t_1, \ldots, t_n)$. Both let expressions and abstractions can be inlined (with the corresponding substitution) to obtain a process with the basic syntax outlined above.

$$
\begin{aligned}
\mathsf{Initiator}(A, sk_A) \triangleq\ & c(\langle B, pk_B\rangle).\ \textbf{new}\ n_A.\ \bar{c}\langle \mathsf{a_enc}(\langle A, n_A\rangle, pk_B)\rangle. \\
& c(x).\ \mathsf{let}\ m_2 = \mathsf{a_dec}(x, sk_A)\ \mathsf{in} \\
& \mathsf{if}\ \pi_1(m_2) = n_A\ \mathsf{then}\ \big(\mathsf{let}\ n_B = \pi_2(m_2)\ \mathsf{in}\ \bar{c}\langle \mathsf{a_enc}(n_B, pk_B)\rangle.0\big) \\
& \mathsf{else}\ 0
\end{aligned}
$$

$$
\begin{aligned}
\mathsf{Responder}(B, sk_B) \triangleq\ & c(y).\ \mathsf{let}\ m_1 = \mathsf{a_dec}(y, sk_B)\ \mathsf{in} \\
& \mathsf{let}\ A' = \pi_1(m_1)\ \mathsf{in}\ \mathsf{let}\ n_A = \pi_2(m_1)\ \mathsf{in}\ \textbf{new}\ n_B. \\
& c(\langle A'', pk_A\rangle).\ \mathsf{if}\ A'' = A'\ \mathsf{then}\ \bar{c}\langle \mathsf{a_enc}(\langle n_A, n_B\rangle, pk_A)\rangle.0\ \mathsf{else}\ 0 \\
& c(z).\ \mathsf{let}\ m_3 = \mathsf{a_dec}(z, sk_B)\ \mathsf{in}\ \mathsf{if}\ m_3 = n_B\ \mathsf{then}\ \bar{c}\langle \mathsf{ok}\rangle.0\ \mathsf{else}\ 0
\end{aligned}
$$

$$
\mathsf{Principal} \triangleq \textbf{new}\ A.\ \textbf{new}\ sk_A.\ \bar{c}\langle\langle A, \mathsf{pk}(sk_A)\rangle\rangle.\big(\ !\mathsf{Initiator}(A, sk_A)\ \mid !\mathsf{Responder}(A, sk_A)\ \big)
$$

$$
\mathsf{System} \triangleq\ !\mathsf{Principal}
$$

Above, the Principal process loops on both roles, Initiator and Responder, with a fixed identity A (i.e., principal A runs unbounded sessions of both roles). Because the top-level System replicates Principal, we also get unbounded principals. Initiator retrieves an identity B and public key pk_B along c (sent by Principal) and starts a new session with fresh nonce n_A, sending the first message along the network channel with $\bar{c}\langle \mathsf{a_enc}(\langle A, n_A\rangle, pk_B)\rangle$. Then, it takes the response (from B) as input from the network with $c(x)$, decrypts the incoming message, and checks if the response correctly returns n_A. If so, it retrieves B's challenge nonce n_B and returns its encryption along $\bar{c}$. If decryption fails or if n_A is not returned, the guard $\pi_1(m_2) = n_A$ will fail (as $\pi_1(m_2) \neq_E n_A$) and the initiator will stall at the empty process rather than sending the last

message. One can trace the Responder's logic playing role B in an analogous fashion. Notice that we are emitting a special term ok to denote that the Responder successfully finished a protocol session to make it observationally distinguishable from the case where the session was unsuccessful. This idea will be expanded in Section 8 with the discussion of *events*.

The above encoding has an attack not present in the informal protocol description. Since the identity to public key mapping $\langle A, \mathsf{pk}(sk_A)\rangle$ is sent along the public network channel (by Principal), the attacker can simply replace A's public key with his own, i.e. $\langle A, \mathsf{pk}(sk_I)\rangle$. Modern cryptosystems prevent this attack with a separate mechanism involving *certificates* and *certificate validation*, but this is out of scope for this document, and ID to public key mappings are usually assumed secure for the purposes of protocol analysis. To disallow this attack, one must extend the applied π-calculus syntax and semantics to support additional features—either a private channel to prevent adversarial tampering, or a global constant providing an ID to public key mapping that can be directly referenced by protocol participants.

Security properties. Security properties in the applied π-calculus setting are traditionally phrased in terms of *observational equivalences* between processes, where observational equivalence is defined in terms of *barbs*. Intuitively, a channel c is a *barb* for process P if P is capable of sending some message along c. More formally, channel c is a *barb* for process P if, for some term M, $P \to^* C[\bar{a}\langle M\rangle]$ for some context C such that a is not bound in C. We denote "c is a barb for P" as $P \Downarrow c$. Then, we can define *observational equivalence* ($\approx$) as the largest symmetric relation between closed extended processes such that $P \approx Q$ implies

(*i*) if $P \Downarrow c$ for some c, then $Q \Downarrow c$;

(*ii*) if $P \to^* P'$, then there exists some Q' such that $Q \to^* Q'$ and $P' \approx Q'$; and

(*iii*) $C[P] \approx C[Q]$ for all contexts C.

Now consider an abstraction $P(s)$ for some process P that is supposed to keep s secret. We say that P satisfies the (strong) secrecy of s if for all s and s', $P(s) \approx P(s')$.

While we did not explicitly encode attacker rules as in Figure 7, the attacker is played by the arbitrary context C in point (*iii*) above. Consider the strong secrecy of s in this small process: $P(s) \triangleq \mathsf{if}\ s = c\ \mathsf{then}\ \bar{c}\langle \mathsf{ok}\rangle\ \mathsf{else}\ \bar{c}\langle \mathsf{ko}\rangle$ (assume ok, ko, and c are concrete constants). Here, $P(s)$ branches on s, but it never reveals it on the channel. Here, context C can distinguish c from any other value for s, where $C[\Box] \triangleq (\Box \mid c(M).\mathsf{if}\ M = \mathsf{ok}\ \mathsf{then}\ \bar{d}(\mathsf{ok})\ \mathsf{else}\ 0.)$ In general, the adversary can perform always perform *tests* in the guard of if expressions and emit a barb in one case and not in the other. Here, $C[P(c)]$ gives a barb on d, while $C[P(c')]$ (with $c \neq c'$) does not give a barb on d, hence breaking the observational equivalence. Note that an evaluation context C intuitively has the same capabilities as a symbolic adversary shown in Figure 1—the active substitutions give the context access to public terms; the adversary can apply function symbols to terms in scope; the adversary can sample fresh terms with **new**; the adversary can use the guard to if expressions to reason equationally; and the adversary *cannot* directly reference terms that are out of scope (e.g., secrets).

Protocol trace. From the above semantics, one could define a protocol trace in terms of a sequence of small steps according to $\to$. However, this would only capture *honest traces*, that is, without adversarial influence. Adversarial influence is only present if we consider an arbitrary context, as in the definition of observational equivalence (see point (*iii*) above). However, universally quantifying over all possible contexts (as in (*iii*)) complicates reasoning, and hence a small-step semantics that inherently captures the interaction of a process with an arbitrary environment (along with its own notion of equivalence) is desirable. In fact, this is why observational equivalence is not mentioned among the equivalence properties in Section 5. To capture protocol traces that "bake in" adversarial influence, we need to consider an alternative small-step semantics.

7.2.1 Alternative small-step semantics

In the *labeled operational semantics*, we define a relation over *configurations* that inherently captures an arbitrary context (i.e., adversarial influence).

$$(\bar{c}\langle u\rangle.P \cup^{\#} \mathcal{P}, \phi, \sigma) \xrightarrow{\bar{c}\langle w\rangle} (P \cup^{\#} \mathcal{P}, \phi \cup \{w \mapsto u\sigma\}, \sigma) \quad \textsc{Out}$$

where $w \in \mathcal{W}$ is fresh and $u\sigma$ is a message

$$(c(u).P \cup^{\#} \mathcal{P}, \phi, \sigma) \xrightarrow{c(R)} (P \cup^{\#} \mathcal{P}, \phi, \sigma \cup \sigma') \quad \textsc{In}$$

where R is a recipe such that $R\phi\downarrow = (u\sigma)\sigma'$,
$R\phi\downarrow$ is a message, and
the domain of σ' is equal to the set of variables in $u\sigma$.

$$(\{0\} \cup^{\#} \mathcal{P}, \phi, \sigma) \xrightarrow{\tau} (\mathcal{P}, \phi, \sigma) \quad \textsc{Null}$$

$$(\{P \mid Q\} \cup^{\#} \mathcal{P}, \phi, \sigma) \xrightarrow{\tau} (\{P, Q\} \cup^{\#} \mathcal{P}, \phi, \sigma) \quad \textsc{Par}$$

Labeled small-step rules for message output, message input, null processes, and parallel composition. Each configuration is comprised of a multiset of processes, a substitution denoting adversary knowledge, and a substitution defining variables for honest processes.

Figure 10: Simplified alternative small-step semantics

A *configuration* is a three-tuple $(\mathcal{P}, \phi, \sigma)$ where $\mathcal{P}$ is a multiset of processes and ϕ and σ are substitutions. Intuitively, ϕ represents the adversary's knowledge, and σ stores the terms referenced by the variables in honest processes. We define the relation $\xrightarrow{\alpha}$ with the following rules; we rely on a set of *attacker variables* $\mathcal{W} \subset V$ used to store terms learned by the attacker and a set of *attacker names* $\mathcal{N}_{\mathcal{W}} \subset \mathcal{N}$ to represent public names and fresh nonces sampled by the adversary. Here, *messages* denote terms in $\mathcal{T}(\Sigma_c, \mathcal{N})$ (ground terms), and *recipes* denote terms in $\mathcal{T}(\Sigma_c, \mathcal{W} \cup \mathcal{N}_{\mathcal{W}})$ (adversary terms).

Intuitively, Out adds to the adversary's knowledge by extending ϕ, and In extends the environment such that the incoming message referenced by u maps to some term R chosen by the adversary from terms in the adversary's knowledge. For simplicity of presentation, note that the above semantics considers a slightly different (simplified) version of the applied π-calculus without replication or equality tests. In this setup, freshness is handled semantically (through $\mathcal{N}$, $\mathcal{N}_{\mathcal{W}}$, and the rules above), and equality tests are performed via pattern matching on the second argument to out (in this setting, we exchange terms, not just variables).

We denote the transitive closure of $\xrightarrow{\alpha}$ as $\xrightarrow{\alpha_1,\ldots,\alpha_n}$. Given $\mathcal{K} \xrightarrow{\alpha_1,\ldots,\alpha_n} \mathcal{K}'$, we can extract a *trace* tr from $\alpha_1, \ldots, \alpha_n$ by removing each τ label; we use $\mathcal{K} \xrightarrow{\pi\colon \mathsf{tr}} \mathcal{K}'$ to denote that $\mathcal{K}$ computes to $\mathcal{K}'$ with trace tr. We define the set of traces starting from configuration $\mathcal{K}$ as follows:

$$\mathsf{traces}(\mathcal{K}) = \{\mathsf{tr} \mid \mathcal{K} \xrightarrow{\pi\colon \mathsf{tr}} (\mathcal{P}', \phi', \sigma') \text{ for some configuration } (\mathcal{P}', \phi', \sigma')\}$$

We employ this more concrete notion of applied π-calculus traces later in Section 9.

Relationship between the semantics. Abadi and Fournet [3] extend the labeled small-step semantics to the entire applied π-calculus, define a notion of equivalence for the labeled small-step semantics called *labeled bisimilarity* denoted $\equiv_l$, and prove that observational equivalence is equal to labeled bisimilarity (i.e., $\equiv = \equiv_l$).

7.3 Other approaches

The last main approach to formalizing security protocols in the symbolic model is through *strand spaces* [44], which take a more algebraic approach. The formalism lends itself to theoretical arguments (i.e., it has been proven equivalent to the multiset rewriting approach [22] for some notion of equivalence). However, it suffers in point (*iii*)—the protocol specifications are not directly executable as in multiset rewriting and the applied π-calculus.

Another approach that (while interesting) never gained much traction is the *inductive approach* of Paulson [66]. Here, they define protocols inductively as sets of traces, and properties are proved by induction on traces using Isabelle. However, the approach does not lend itself to automatically finding attack traces (a failure to produce an Isabelle proof does not always naturally suggest a counterexample).

In SATMC [6] the protocol verification problem is phrased in terms of Boolean satisfiability queries; however, this inherently involves putting a fixed bound on the term size, the cardinality of the data items D, and the number of sessions and principals considered.

Another notable approach is to achieve security through typing—that is, design a type system such that the program will only type check if the program satisfies given security properties; notable examples are F7 [14] and DY* [13]. However, to achieve termination in type checking, these tools consider an overapproximation of the adversary's knowledge $\mathcal{K}$, and hence the type checker may reject programs that are actually secure.

8 Mechanized reasoning in the Symbolic Model

We discuss a naive approach to the mechanization of reasoning, and why it fails. Then, we give a general overview of Tamarin and ProVerif's approach to mechanized reasoning. While we outline general strategies, gory details are outside the scope of this document.

8.1 Naive approach

A general-purpose model checker takes a system model M and a property φ and attempts to prove that φ is valid in M, i.e., $M \models \varphi$. Many modern model checkers can reason about infinite-state systems [21, 23], and hence one would not need to impose a bound on term side, D, sessions, or principals as in [6]. However, there is no straightforward way to use general-purpose model checkers for security protocol proofs: If cryptographic constructs (like encryption and decryption) are ignored (e.g., replacing encrypted messages with plaintexts), then spurious counterexamples are raised; if one attempts to incorporate cryptographic constructs, then one must incorporate adversarial knowledge and capabilities into the system model and/or security property. The former approach is still possible, but it relies on abstraction-refinement techniques that diminish automation potential or miss attacks [50, 51, 55]. The latter approach is difficult, error-prone, and often computationally infeasible without dedicated algorithms. Because of this, dedicated approaches have been developed for cryptographic protocol verification. Today, the most popular approaches are based on developing algorithms for mechanizing the reasoning in the multiset rewriting and applied π-calculus formalisms, as discussed in Sections 7.1 and 7.2.

8.2 Tamarin

An example tool mechanizing reasoning with formal foundations in multiset rewriting is Tamarin [60]. From the input protocol R, set of adversary rules Adv, and the negation of the security property $\neg\varphi$, Tamarin creates a constraint system, i.e., a set of constraints S (which is a subset of the set of all constraints $\mathcal{S}$). Tamarin iteratively applies constraint solving rules from its core calculus (i.e., rules defining a transition relation $\rightsquigarrow \subseteq \mathcal{P}(\mathcal{S}) \times \mathcal{P}(\mathcal{S})$) until either the current set of constraints is trivially satisfiable (resulting in a concrete attack trace), or it is trivially unsatisfiable (meaning that the property is valid in the protocol). The details of $\rightsquigarrow$ will not be discussed in this report; however, we outline a few technical challenges.

Unification modulo E. Formally, Tamarin performs *unification modulo* E. Two terms t_1 and t_2 are *unifiable modulo* E if there exists some substitution σ such that $t_1\sigma =_E t_2\sigma$; this σ is called a *unifier*. In general, we are interested in *most general unifiers* (MGUs). A unifier σ of terms t_1 and t_2 is *most general* if for all unifiers σ' of t_1 and t_2, there exists a substitution θ such that $\sigma' = \sigma \circ \theta$. We use $mgu(t_1, t_2)$ to denote the most-general unifier of terms t_1 and t_2. Intuitively, we use MGUs because they impose the fewest constraints. For example, suppose we want to unify $\mathsf{enc}(x, k)$ with $\mathsf{enc}(g(y), k)$ with $E = \emptyset$. Then an example MGU is $\sigma = \{x \mapsto g(y)\}$. A unifier that is not most general is $\sigma' = \{y \mapsto a, x \mapsto g(a)\}$; we can verify that σ is more general than σ', since $\sigma' = \sigma \circ \{y \mapsto a\}$. In the context of searching for attacks, we prefer the more general unifier (σ) because the less general unifier (σ') imposes an extra constraint $y = a$

that is not logically entailed by the equality of the two terms. Unification modulo E is a difficult problem, but it can be decidable depending on the properties of E. If $E = \emptyset$ (and all terms are first-order terms), then the problem is called *syntactic unification* or simply *unification*. Syntactic unification of first-order terms (yielding an MGU, unique up to renaming) is efficiently decidable [59], and hence we would like to reduce unification modulo E to syntactic unification. To do so, we need to borrow some insights from the *term rewriting* literature.

Term rewriting preliminaries. A *term rewriting rule* is a pair (l, r) (often written $l \to r$) with $l, r \in \mathcal{T}(\Sigma, V)$, and a *term rewriting system* is a set of term rewriting rules. A term rewriting system $\mathcal{R}$ induces the binary relation $\to_R$ capturing all possible applications of rewrite rules. More formally, $t_1 \to_{\mathcal{R}} t_2$ if there is a position p in t_1, rewrite rule $l \to r \in \mathcal{R}$, and substitution σ such that $t_1|_p = l\sigma$ and $t_1[r\sigma]_p = t_2$ (intuitively, for term t_1, find a subterm that matches the left-hand side of the rewrite rule and replace it with the right-hand side of the rewrite rule, with the corresponding substitution). A rewriting system is *terminating* if there is no infinite sequence of terms with $t_i \to_{\mathcal{R}} t_{i+1}$, and it is *confluent* if for all terms t, s_1, and s_2 with $t \to^*_{\mathcal{R}} s_1$ and $t \to^*_{\mathcal{R}} s_2$, there exists a term t' such that $s_1 \to^*_{\mathcal{R}} t'$ and $s_2 \to^*_{\mathcal{R}} t'$. Confluence is sometimes called the *diamond property*; it intuitively means that a rewrite system cannot find two differing paths for a given term. A rewrite system is *convergent* if it is both confluent and terminating. Convergence implies the existence of *unique normal forms*, obtained by rewriting a term until no rewrite rule applies. The normal form of a term t in rewrite system $\mathcal{R}$ is denoted $t{\downarrow_{\mathcal{R}}}$. We take advantage of theoretical results in term rewriting by orienting equations in equational theories from left to right and viewing them as rewriting rules comprising a rewrite system. Due to this, we call an equational theory *convergent* if it can be posed as a convergent rewrite system.

Finite variant property. A convergent equational theory E has the *finite variant property* if for any term t, one can compute a finite set of pairs $(t_1, \theta_1), \ldots, (t_n, \theta_n)$, called *variants* of t, such that for each i we have $t_i = t\theta_i{\downarrow_E}$, and such that for every substitution σ, there exists an index i and a substitution σ' with $\sigma = \theta_i \circ \sigma'$ and $t\sigma{\downarrow_E} = t_i\sigma'$. Intuitively, one can view $(t_1, \theta_1), \ldots, (t_n, \theta_n)$ as a finite set of variants of term t that characterize the different "shapes" it can take (where each θ_i records how we bridge from t to t_i). The finite variant property is useful because it can help us reduce unification modulo E to syntactic unification, as in the following algorithm.

Algorithm sketch for unification modulo E. Suppose we are given t_1 and t_2, and we want to find σ such that $t_1\sigma =_E t_2\sigma$. We proceed as follows.

Assume E is convergent and has the finite variant property.

(1) Rename variables in t_2 so the variable sets of t_1 and t_2 are disjoint.

(2) Compute the variants $(t_{1,1}, \theta_{1,1}), \ldots, (t_{1,n}, \theta_{1,n})$ for t_1 and the variants $(t_{2,1}, \theta_{2,1}), \ldots, (t_{2,m}, \theta_{2,m})$ for t_2. One can do so as in [74] using a technique called *folding variant narrowing*.

(3) Perform syntactic unification for each pair $(t_{1,i}, t_{2,j})$, each failing or returning an MGU $\mu_{i,j}$.

(4) For each successful unification, return the composed substitution $\sigma_{i,j} = \theta_{1,i} \circ \theta_{2,j} \circ \mu_{i,j}$.

Then, Tamarin can use the $\sigma_{i,j}$s computed above to case split in the search.

Example of unification modulo E. Consider the unification problem for terms $\mathsf{dec}(x, k)$ and y with the equational theory $\{\mathsf{dec}(\mathsf{enc}(m, k), k) = m\}$. We run the algorithm as follows.

E is indeed convergent and has the finite variant property.

(1) The variables of $\mathsf{dec}(x, k)$ and y are already disjoint.

(2) y has a single variant, $(y, \theta_{1,1} = \{\})$. $\mathsf{dec}(x, k)$ has two variants, $(m, \theta_{2,1} = \{x \mapsto \mathsf{enc}(m, k)\})$ and $(\mathsf{dec}(x, k), \theta_{2,2} = \{\})$, intuitively corresponding to the case where x is decryptable and the case where it isn't (respectively).

(3) We syntactically unify two pairs (y, m) and $(y, \mathsf{dec}(x, k))$, yielding substitutions $\mu_{1,1} = \{y \mapsto m\}$ and $\mu_{1,2} = \{y \mapsto \mathsf{dec}(x, k)\}$.

(4) Return $\theta_{1,1} \circ \theta_{2,1} \circ \mu_{1,1} = \{x \mapsto \mathsf{enc}(m,k), y \mapsto m\}$ and $\theta_{1,1} \circ \theta_{2,2} \circ \mu_{1,2} = \{y \mapsto \mathsf{dec}(x,k)\}$.

Tamarin's property specification language. Tamarin's property specification logic is a variant of first-order logic, where sorts have fixed domains and formulas are interpreted over valuations and multiset rewriting traces. We introduce a new sort *tmp* for temporal variables, and we fix a domain for each sort: for *msg* it is $\mathcal{T}(\Sigma, \mathcal{N})$; for *fr* and *pub* it is $\mathcal{N}$; for *tmp* it is $\mathbb{N}$. A function $\theta :: V \to \mathbb{N} \cup \mathcal{T}(\Sigma, \mathcal{N})$ is a *valuation* if it respects sorts, and we overload notation and allow $\theta(t)$ to represent the application of the homomorphic extension of θ to term t (and resp., fact f). The syntax is captured in the following grammar; each v refers to a variable, each t a term, and each f an action fact.

$$\begin{array}{rl} \mathit{atom} & ::= \bot \mid t_1 = t_2 \mid i^{\text{TMP}} < j^{\text{TMP}} \mid i^{\text{TMP}} = j^{\text{TMP}} \mid f@i^{\text{TMP}} \\ \varphi & ::= \mathit{atom} \mid \varphi_1 \wedge \varphi_2 \mid \neg\varphi \mid \exists v.\varphi \end{array}$$

Q for Cesare—intuition for why guardedness helps make reasoning easier? And for loss in expressivity?

Tamarin requires that properties must be *guarded sentences*. A *sentence* is a formula with no free variables, and *guardedness* requires that all quantifiers must be of the form $\exists x_1. \ldots . \exists x_n. f@i \wedge \varphi$ or $\forall x_1. \ldots . \forall x_n. f@i \Rightarrow \varphi$, where $x_1, \ldots, x_n \in fv(f@i)$. Intuitively, guarded quantification eases reasoning by restricting the possible instantiations for quantified variables (i.e., only those instantiations that satisfy some predicate), as well as restricting interactions between quantified variables (i.e., quantified variable occurrences in φ must be related by some predicate). While the guarded fragment is less expressive (not every sentence is translatable to a guarded sentence), it is expressive enough to specify all standard security properties.

Now, we can define a satisfaction relation $tr, \theta \models_E \varphi$ which means that trace tr with valuation θ satisfies formula φ, modulo equational theory E. Below, $idx(tr)$ refers to the index set of trace tr, and $\in_E^{\#}$ refers to multiset membership modulo E.

$(tr, \theta) \models_E$	$\bot$	never
$(tr, \theta) \models_E$	$f@i$	if $\theta(i) \in idx(tr)$ and $\theta(f) \in_E^{\#} tr_{\theta(i)}$
$(tr, \theta) \models_E$	$i^{\text{TMP}} < j^{\text{TMP}}$	if $\theta(i^{\text{TMP}}) < \theta(j^{\text{TMP}})$
$(tr, \theta) \models_E$	$i^{\text{TMP}} = j^{\text{TMP}}$	if $\theta(i^{\text{TMP}}) = \theta(j^{\text{TMP}})$
$(tr, \theta) \models_E$	$t_1 = t_2$	if $\theta(t_1) = \theta(t_2)$
$(tr, \theta) \models_E$	$\neg\varphi$	if $(tr, \theta) \not\models_E \varphi$
$(tr, \theta) \models_E$	$\varphi_1 \wedge \varphi_2$	if $(tr, \theta) \models_E \varphi_1$ and $(tr, \theta) \models_E \varphi_2$
$(tr, \theta) \models_E$	$\exists v.\varphi$	if there exists u in the domain of v such that $(tr, \theta[v \mapsto u]) \models_E \varphi$

The above semantics is standard; the most interesting case is that of $f@i$. It states that $f@i$ is satisfied if action fact f is in the trace at index i (with both f and i interpreted according to the valuation θ). Note that one can support universal quantification and the rest of the standard connectives by desugaring to existential quantification, conjunction, and negation. From the standard satisfaction relation, one can say that a security property φ *holds* in a model R if $(tr, \theta) \models_E \varphi$ for every evaluation θ and every trace tr of R.

Standard security properties in Tamarin. Below we present how to formalize standard security properties in Tamarin's logic; variable sort annotations are omitted for brevity.

- *Weak secrecy*:

$$\forall x.\ \forall i.\ \mathsf{Secret}(x)@i \Rightarrow \neg(\exists j.\mathsf{K}(x)@j)$$

  We discussed this in Section 7.1.

- *Non-injective agreement*:

$$\forall A.\forall B.\forall t.\forall i.\mathsf{Accept}(A, B, t)@i \Rightarrow (\exists j.\mathsf{Sent}(B, A, t)@j \wedge j < i)$$

  This closely aligns with the natural language definition of non-injective agreement: if B accepted term t (apparently) from A, then A sent term t (apparently) to B at some earlier time.

- *Injective agreement*:

$$\forall A.\forall B.\forall t.\forall i.\mathsf{Accept}(A, B, t)@i \Rightarrow (\exists j.\mathsf{Sent}(B, A, t)@j \wedge j < i \wedge \\ \neg(\exists A_2.\exists B_2.\exists i_2.\mathsf{Accept}(A_2, B_2, i_2)@i_2 \wedge \neg(i_2 = i)))$$

The formalization of injective agreement is the same as non-injective agreement, save for the extra conjunct on the second line. It states that if A accepted t from B, then A sent t to B, *and also* there was not another distinct accept event for the same term t. At first glance, the property may seem too strong—for example, what if A sends t twice, and B accepts t twice? It falsifies this property, but should not violate the textbook definition of injective agreement (from Section 5). Although this trace technically does not violate the textbook definition, one can easily construct another trace from it that *does* violate it—simply replace the second honest send with an adversarial replay. Hence, each term sent by an honest participant must include some notion of freshness (e.g., via a nonce or counter) if a protocol hopes to satisfy injective agreement.

One can specify *diff equivalence*, but not through the property specification logic, as it only captures trace properties. To do so, one annotates the input rewriting system with $\mathsf{diff}(t_1, t_2)$ terms. Then, one can run Tamarin in `diff` mode, and Tamarin will auto-generate two versions of the protocol, one with t_1 substituted for $\mathsf{diff}(t_1, t_2)$ (for each occurrence of $\mathsf{diff}(t_1, t_2)$), and one with t_2 substituted for $\mathsf{diff}(t_1, t_2)$ (for each occurrence of $\mathsf{diff}(t_1, t_2)$). From these two protocols, Tamarin then attempts to determine their diff equivalence. For example, consider the single-rule protocol

$$\{\mathsf{Fr}(ltk), \mathsf{Fr}(a), \mathsf{Fr}(b)\} \rightarrow \{\mathsf{Out}(\mathsf{pk}(ltk)), \mathsf{Out}(a), \mathsf{Out}(\mathsf{a_enc}(\mathsf{diff}(a, b), \mathsf{pk}(ltk))\}$$

From this, Tamarin would generate two protocols, each a single rule where the first one substitutes a for $\mathsf{diff}(a, b)$, and the second where b is substituted for $\mathsf{diff}(a, b)$. Intuitively, these protocols are not diff equivalent because the adversary can tell whether a or b was encrypted by computing the encryption of a himself and comparing it to the ciphertext. Hence, even though b is secret, some information about b is leaked.

8.3 ProVerif

An example tool mechanizing reasoning with formal foundations in applied π-calculus is ProVerif [16]. Instead of taking a small-step operational semantics (as in Section 7.2), ProVerif implements a *translational semantics* Q for Cesare—is this the right terminology? where the input process and adversarial influence are translated to a set of *Horn clauses*.

8.3.1 Horn clause preliminaries

We start with a general discussion of Horn clauses.

Horn clauses. Each *Horn clause* is a first-order logic formula of the form $P_1 \wedge \cdots \wedge P_n \Rightarrow P_{n+1}$ (or just a unit clause P), where each P_i is a predicate. Note that the free variables in a Horn clause are all universally quantified, but the quantifiers are typically left implicit presentationally. In this setting, we consider the *predicate signature* $\Sigma_p = \{(\mathsf{Attacker}, msg), (\mathsf{Message}, msg \times msg)\}$ and predicate arguments from $\mathcal{T}(V, D)$. Intuitively, $\mathsf{Attacker}(t)$ denotes that the term t is in the adversary's knowledge base, and $\mathsf{Message}(m, c)$ denotes that message m is sent on channel c. Note that for convenience, we may use notation $H \Rightarrow C$ where H is a multiset, which denotes clause $H_1 \wedge \cdots \wedge H_n \Rightarrow C$. Here, each H_i is a multiset element of H (including repetitions).

Horn clause semantics. We only consider models with a domain of $\mathcal{T}(\Sigma_c, D)$ that map function and constant symbols to the corresponding constructor symbols. In first-order logic, these are called *Herbrand models*. Herbrand models of a given set of formulas can only differ in the interpretation they give to the predicate symbols, and one can encode a Herbrand model as the set of ground atomic formulas that are true in the model. Then, the *least Herbrand model* of a set of Horn clauses ψ is a Herbrand model $\mathcal{H}$ such that for every other Herbrand model $\mathcal{H}'$ of ψ, $\mathcal{H} \subseteq \mathcal{H}'$. Note that least Herbrand models may not exist in general, but they always exist in the context of Horn reasoning [76].

Horn clause example. Consider the signature $\Sigma_c = \{(f, msg), (c_1, \rightarrow msg), (c_2, \rightarrow msg)\}$ with predicate symbols p_1 and p_2, and consider the Horn clause set $\{p_1(f(c_1)), p_1(f(x)) \Rightarrow p_2(f(x))\}$. Then, an example Herbrand model is $\{p_1(f(c_1)), p_2(f(c_1)), p_1(f(c_2))\}$, and the least Herbrand model is $\{p_1(f(c_1)), p_2(f(c_1))\}$.

Semantics of negation. In the previous example, $p_1(f(c_2))$ exists in some Herbrand model, but not in the *least* Herbrand model. Logic programming systems with *negation as failure* would treat $\neg p_1(f(c_2))$ as

derivable, as $p_1(f(c_2))$ is not a member of the least Herbrand model. However, this departs from the typical semantics of negation in first-order logic, as it is not monotonic—one could add another Horn clause to the set to make $p_1(f(c_2))$ derivable (in the least semantics), invalidating $\neg p_1(f(c_2))$. Negation as failure complicates reasoning, so ProVerif's Horn clause-based property specification language does not support negation. Rather, it only supports $\bot$, denoting falsity.

8.3.2 ProVerif's algorithm

Next, we give a general outline of ProVerif's algorithm.

Saturation by resolution. Using the set of Horn clauses generated from the input processes, ProVerif iteratively derives new Horn clauses until reaching a fixpoint in a process called *saturation* by *resolution*. Here, *resolution* refers to the inference rule below (used to derive new clauses), and *saturation* refers to the fact that resolution is applied repeatedly until reaching a fixpoint. Clauses to resolve (i.e., perform a resolution step on) are selected heuristically.

$$\frac{H \Rightarrow C \quad F \wedge H' \Rightarrow C' \quad \sigma \text{ is the MGU of } C, F}{\sigma H \wedge \sigma H' \Rightarrow \sigma C'} \textsc{Res}$$

For example, one could resolve $\mathsf{Attacker}(k)$ with $\mathsf{Attacker}(\mathsf{enc}(x_1, x_2)) \wedge \mathsf{Attacker}(x_2) \Rightarrow \mathsf{Attacker}(x_1)$ to derive $\mathsf{Attacker}(\mathsf{enc}(x_1, k)) \wedge \mathsf{Attacker}(k) \Rightarrow \mathsf{Attacker}(x_1)$ with $\sigma = \{x_2 \mapsto k\}$. Saturation by resolution is amenable to several optimizations— for example, *tautologies* of the form $H \wedge \cdots \Rightarrow H$ are eliminated, as well as *subsumed clauses*. Clause $H \Rightarrow C$ is *subsumed by* clause $H' \Rightarrow C'$ if there exists σ such that $H'\sigma \subseteq^{\#} H$ and $C'\sigma = C$ (i.e., we can achieve the same conclusion with fewer hypotheses). So, by stating that resolution proceeds until "reaching a fixpoint," we mean that it proceeds until it is not possible to derive a new clause, where *new* denotes that the clause is not deemed redundant by some optimization.

When possible, ProVerif simulates equational theories in the form of *destructor rules* $\mathcal{D}$, each of the form $g(t_1, \ldots, t_n) \longrightarrow t$, where $(g, n) \in \Sigma_d$ and $t_1, \ldots, t_n \in \mathcal{T}(\Sigma_c, V)$ (for example, $\mathsf{dec}(\mathsf{enc}(m, k), k) \longrightarrow m$). These destructor rules have a natural translation to ProVerif's Horn clause encoding (see case (2) below). The differences between destructor rules and equational theories may seem superficial, but they are meaningful and will be discussed further in Section 9.

ProVerif's direct support for equational theories is limited to those than can be partitioned into a set of convergent equations and a set of *linear* equations (where each variable is used at most once on the left-hand side and at most once on the right-hand side). Broadly speaking, ProVerif "compiles away" the equational theory by transforming the clause set such that Horn clause resolution can still proceed with syntactic unification [17]. In principle, ProVerif could augment its unification procedure within Horn clause resolution to support richer equational theories as in Tamarin. However, this clashes with ProVerif's goal of attaining a higher level of automation.

Clauses for the adversary. ProVerif encodes adversarial capabilities into Horn clauses with the following rules, which are parametrized by the signature $\Sigma = \Sigma_c \cup \Sigma_d$:

$$\{\mathsf{Attacker}(t_1), \ldots, \mathsf{Attacker}(t_n) \Rightarrow \mathsf{Attacker}(f(t_1, \ldots, t_n)) \mid (f, n) \in \Sigma_c\} \cup \quad (1)$$
$$\{\mathsf{Attacker}(t_1) \wedge \cdots \wedge \mathsf{Attacker}(t_n) \Rightarrow \mathsf{Attacker}(t) \mid (g, n) \in \Sigma_d \wedge g(t_1, \ldots, t_n) \longrightarrow t \in \mathcal{D}\} \cup \quad (2)$$
$$\{\mathsf{Message}(m, c) \wedge \mathsf{Attacker}(c) \Rightarrow \mathsf{Attacker}(m)\} \cup \quad (3)$$
$$\{\mathsf{Attacker}(m) \wedge \mathsf{Attacker}(c) \Rightarrow \mathsf{Message}(m, c)\} \quad (4)$$

Above, cases (1) and (2) correspond to the adversary's ability to construct and destruct messages, respectively. Case (3) denotes that the adversary learns messages that are sent along adversary-controlled channels. Case (4) denotes that the adversary can send messages along channels (so long as the adversary knows both the message and the channel).

Additionally, Horn clauses containing $\mathsf{Message}(.,.)$ predicates are generated to also capture the protocol rules—this translation is more complicated, and the internal workings of ProVerif are not the focus of this document. However, the translation rules can be found in Figure **??** in the appendix.

For secrecy properties, ProVerif checks if $\mathsf{Attacker}(s)$ is derivable (from the protocol and adversary rules) for some secret s, using an algorithm based on saturation by resolution [16]. A general takeaway is that with ProVerif's Horn clause semantics, secrecy and authentication properties can also be framed naturally in terms of *reachability* rather than observational equivalence, increasing the automation potential.

ProVerif's property specification language and events. Rather than providing a generic temporal logic for property specification, ProVerif supports three types of queries: (*i*) derivability queries, (*ii*) correspondence queries, and (*iii*) equivalence queries. Derivability queries are conjunctions of predicates $P_1 \wedge \cdots \wedge P_n$. Similar to Tamarin, ProVerif has a property specification language to encompass cases (*i*) and (*ii*), and it has a separate equivalence checking mode for case (*iii*). For derivability queries, ProVerif performs saturation by resolution on the input clause set (generated from the protocol and adversary rules), and then checks for the derivability of each predicate in the query. The predicates must only take terms that are constructed with free names. Correspondence queries, however, are a bit more complicated. To help capture correspondences, ProVerif uses an analogue of Tamarin's action facts, called *events*, to record events on the trace. ProVerif extends the process syntax to also include $\mathsf{event}\ id(t_1, \ldots, t_n)$ (e.g., $\mathsf{event}\ \mathsf{Accept}(A,\ B,\ t)$), and also performs a corresponding extension of the Horn clause translation to include Event predicates. Here, id is called an *event name*. As in Tamarin, the correspondence language supports natural numbers and the annotation of events with a timepoint as in $e(t_1, \ldots, t_n)@i$ denoting that event e with terms $t_1, \ldots, t_n$ occurred at timestep i. Armed with events, one can build correspondence properties from the following grammar. Below, each t_i ranges over terms in $\mathcal{T}(\Sigma, D)$.

Q for Cesare—why the syntactic restriction of H? No negation (why not negation as failure)? No existential quantification (how is it supported in prolog)? how do these restrictions help ease reasoning? Reminder—derivability is an overapproximation, maybe this is relevant?

$C ::=$	$F_1 \wedge \cdots \wedge F_n \Rightarrow H$	correspondences
$F ::=$	$P \mid P@i \mid M \bowtie N$ (where $\bowtie \in \{<, \leq, >, \geq, =, \neq\}$)	facts
$H ::=$	$F \mid H \wedge H \mid H \vee H \mid \bot \mid (F \Rightarrow H)$	conclusion formulas
$P ::=$	$\mathsf{Attacker}(t_1) \mid \mathsf{Message}(t_1, t_2) \mid \mathsf{Event}(e) \mid \mathsf{Inj\text{-}Event}(e)$	atomic predicates
$e ::=$	$id(t_1)$ where id is some event name	event instances

First, note that the translation from applied π-calculus to Horn clauses is not exact, so the derivability in ProVerif is over-approximate—for example, if ProVerif derives $\mathsf{Attacker}(m)$, we can only conclude the adversary *may* be able to compute m (we will discuss this further in Section 8.4). For derivability problems, this can lead to spurious attacks, which ProVerif accepts as the cost of a cheaper approximation. However, if $\mathsf{Attacker}(m)$ is not derivable, we can be sure that m is secret, so there is no fear of unsoundly concluding that secrecy holds. Now consider an authentication property $\mathsf{Event}(\mathsf{Accept}(A, B, t)) \Rightarrow \mathsf{Event}(\mathsf{Sent}(B, A, t))$ (i.e., if A accepted term t from B, then B sent term t to A). But, if we conclude that every acceptance has a corresponding send, then the conclusion might not hold in the exact semantics (and may only be due to the approximation). This is more problematic than the case of secrecy, as ProVerif would falsely conclude that the protocol is secure. To address this, ProVerif approximates the antecedents of a correspondence query but recovers an exact analysis for the consequents using a custom algorithm [15]. Additionally, the algorithm implements handling for *injective events* (with $\mathsf{Inj\text{-}Event}$) which impose one-to-one mappings between consequents and antecents (see injective agreement below), as well as temporal variables and constraints.

Could give more details on (1) correspondences, (2) injective correspondences, (3) proverif's labeled operational semantics, (4) semantics of properties wrt ProVerif's semantics

Standard security properties in ProVerif.

- *Weak secrecy*:

$$\mathsf{Attacker}(t)$$

  We check for the derivability of term t by the adversary.

- *Non-injective agreement*:

$$\forall A.\forall B.\forall t.\ \mathsf{Event}(\mathsf{Accept}(A, B, t)) \Rightarrow \mathsf{Event}(\mathsf{Sent}(B, A, t))$$

The property is encoded as a correspondence query between a term t being accepted by A from B and being sent from A to B.

- *Injective agreement*:

$$\forall A.\forall B.\forall t.\ \mathsf{inj\text{-}event}(\mathsf{Accept}(A,B,t)) \Rightarrow \mathsf{inj\text{-}event}(\mathsf{Sent}(B,A,t))$$

  The encoding is a correspondence analogous to non-injective agreement, but we simply replaces the Event keyword with Inj-Event to denote that the correspondence should be injective.

- *Diff equivalence*: ProVerif has a diff(.,.) operator analogously to Tamarin. If any diff is used in the input file, ProVerif automatically checks diff equivalence.

8.4 Soundness and Completeness of Tamarin and ProVerif

Tamarin's constraint solving rules are *attack sound* and *proof sound* in the sense that every attack generated from Tamarin's constraint solving rules is a real protocol trace in Tamarin's semantics (i.e., in the symbolic model), and every property proven by Tamarin's constraint solving rules actually holds in Tamarin's semantics. However, Tamarin is not *attack complete* or *proof complete* in the sense that Tamarin is not guaranteed to find attacks or proofs if they exist [60]. Another way of phrasing Tamarin's soundness is that there are no spurious attacks (or false proofs), and hence the analysis is *exact*.

ProVerif is *not* attack sound (in other words, it sometimes reports spurious attacks). However, it is proof sound. Like Tamarin, it is neither attack complete nor proof complete [16].

ProVerif's unsoundness is not a bug. In fact, ProVerif can always detect (and report) unsoundness *a posteriori* [19], but it may not provide support to alleviate the problem. The following small (but contrived) protocol exhibits ProVerif's unsoundness:

Assume k is known only to A. Each N_i is a fresh nonce.

(1) A sends $\mathsf{s_enc}(\langle N_1, N_3\rangle, k)$ and $\mathsf{s_enc}(\langle N_2, N_3\rangle, k)$ to B.

(2) B sends a message to A. If A receives $\mathsf{s_enc}(\langle x, N_3\rangle, k)$, respond with x.

(3) B sends a message to A. If A receives $\langle N_1, N_2\rangle$, respond with secret s.

We wish to prove the secrecy of s, which indeed holds in the above protocol. In step (1), A sends two fresh nonces to B, each separately encrypted. B does not know k and cannot decrypt. But, B can choose one of the messages to send back to A in step (2), where A serves as a decryption oracle (returning the plaintext). But, by step (3), it is impossible for B (or the adversary) to learn both plaintexts N_1 and N_2 because A will only leak one of them in step (2). Hence, secret s will never be leaked in step (3).

Why does ProVerif report a spurious attack? We can inspect a subset of ProVerif's Horn clause encoding of the protocol:

$\mathsf{Attacker}(c)$	(i)	attacker knows network channel
$\mathsf{Message}(\mathsf{s_enc}(\langle N_1, N_3\rangle, k), c)$	(ii)	protocol rule (1) (part one)
$\mathsf{Message}(\mathsf{s_enc}(\langle N_2, N_3\rangle, k), c)$	(iii)	protocol rule (1) (part two)
$\mathsf{Message}(\mathsf{s_enc}(\langle x, N_3\rangle, k), c) \Rightarrow \mathsf{Message}(x, c)$	(iv)	protocol rule (2)
$\mathsf{Message}(\langle N_1, N_2\rangle, c) \Rightarrow \mathsf{Message}(s, c)$	(v)	protocol rule (3)
...		

Above, Horn clauses (i) denotes that the communication channel is public. Clauses (ii) and (iii) denote that the corresponding messages are sent according to protocol rule (1). Clause (iv) denotes protocol rule (2)—with the given input along c, the given output along c is produced. Analogously, clause (v) denotes protocol rule (3). With these clauses, along with the adversary deduction clauses, ProVerif will iteratively derive new clauses in an attempt to conclude $\mathsf{Attacker}(s)$. From the Horn clause adversary deduction rule along with clauses (ii) and (iii), the adversary can learn both $\mathsf{s_enc}(\langle N_2, N_3\rangle, k)$ and $\mathsf{s_enc}(\langle N_1, N_3\rangle, k)$. Next, ProVerif's algorithm will use clause (iv) *twice*, along with adversary deduction rules, to derive both $\mathsf{Attacker}(N_1)$ and $\mathsf{Attacker}(\mathsf{N}_2)$. Importantly, the Horn clause abstraction loses the linearity of A's protocol

state—once step (2) is executed, in a precise semantics, A would move on to step (3). In contrast, ProVerif's algorithm will reuse clause (iv), representing a repetition of the protocol step.

On the side, notice that N_3 is necessary for the security of the protocol. Say we remove N_3 (i.e., convert all pairs containing N_3 to just the first element of the pair). Here, the attacker *can* trigger step (2) twice, each time corresponding to a *distinct session* of A. But, the existence of N_3 binds step (2) to a particular session. So, if A receives $\{\langle x, N_4\rangle\}_k$ in step (2) for some $N_4 \neq N_3$, the input will not pattern match and B will not respond with x. Hence, the adversary will not be able to construct both pair elements for step (3).

8.5 Automation potential

ProVerif's unsoundness is an intentional feature—while spurious attacks exist, the abstraction lends itself to easier reasoning and hence higher automation potential than Tamarin [31]. In other words, Tamarin pays for the exactness of its analysis through a higher dependence on interactive guidance from the user.

8.6 Support for interaction

ProVerif does not support interactive proof, only interactive simulation (i.e., running the model). On the other hand, Tamarin has an interactive mode with a GUI that supports both simulation and interactive proof, allowing the user to choose which constraint solving transition (from $\rightsquigarrow$) to apply next.

However, the proof techniques employed by ProVerif and Tamarin are both fundamentally opposed to *usable* interaction. In both systems, proofs are monolithic, and large proofs become too overwhelming to be digestible to a human. In the ProVerif world, imagine having a database of 1000 available Horn clauses and being asked to pick the next two to perform a resolution step on. In the Tamarin world, imagine having a proof tree with hundreds of open premises, each with dozens of possible constraint solving steps, and being asked to choose the next step to apply. These tasks are best left to computers, but unfortunately, nontermination becomes a serious obstacle. Expert users can reformulate the encoding or add extra lemmas to help push through difficult proofs. However, this undermines the main goals of automation—easing human burden and allowing tool usage by non-experts.

8.7 Property specification languages

Tamarin supports properties in an expressive fragment of a variant of first-order logic. In contrast, ProVerif's Horn clause resolution algorithm only supports reachability queries in terms of the derivability of a given fact, or in terms of a correspondence query that fits the grammar described earlier.

8.8 Bridge theorems

Finding a principled way to "bridge" between formalisms has two major benefits. First, consider a situation where one has a Tamarin model, but is struggling with nontermination and wants to try using ProVerif's heuristics to achieve automation. Second, consider a situation where one has proved a metatheorem about symbolic model verification (examples will be discussed in Section 10) and wants to apply the metatheorem to their Tamarin model, but the theorem and its proof were stated in terms of the applied π-calculus. In both situations, without a means of bridging the gap between formalisms, there is no easy path forward.

First, one should note that both multiset rewriting and applied π-calculus are Turing complete, and hence some translation between the two, in either direction, is theoretically possible. However, in the context of usable automation, one direction of translation is well-understood, while the other is much more treacherous.

8.8.1 Applied π to MSR

The direction from ProVerif's applied π-calculus to Tamarin's multiset rewriting is easier because ProVerif's applied π-calculus does not contain any features without a natural translation to Tamarin's multiset rewriting.

One can realize this translation with a provably sound transpiler, SAPIC$^+$ [25]. SAPIC$^+$ stands for **S**tateful **A**pplied **P**i **C**alculus, and it is an extension of the applied π-calculus to handle state with the same property specification language as Tamarin. Let $\mathcal{M}$ be a SAPIC$^+$ protocol model, i.e., a process in the applied π-calculus with state, and let φ be a SAPIC$^+$ property. Then, the SAPIC$^+$ paper introduces four functions $[\![.]\!]^{\mathrm{TM}}, [\![.]\!]^{\mathrm{TM}}, [\![.]\!]^{\mathrm{PM}}, [\![.]\!]^{\mathrm{PP}}$, where $[\![.]\!]^{\mathrm{TM}}$ maps SAPIC$^+$ models to Tamarin models (i.e., sets of multiset rewriting rules), $[\![.]\!]^{\mathrm{TP}}$ maps SAPIC$^+$ properties to Tamarin properties, $[\![.]\!]^{\mathrm{PM}}$ maps SAPIC$^+$ models to ProVerif models, and $[\![.]\!]^{\mathrm{PP}}$ maps SAPIC$^+$ properties to ProVerif properties. In the latter two cases, the translation may fail in the case that the SAPIC$^+$ model uses some feature unsupported by ProVerif.

The theoretical contribution is in two main theorems which ensure the correctness of their translation functions. Here, $\models_{\mathrm{S}}$ refers to property satisfaction for SAPIC$^+$ models and properties, $\models_{\mathrm{T}}$ for Tamarin, and $\models_{\mathrm{P}}$ for ProVerif.

Theorem 2. *$\mathcal{M} \models_{\mathrm{S}} \varphi$ if and only if $[\![\mathcal{M}]\!]^{\mathrm{TM}} \models_{\mathrm{T}} [\![\varphi]\!]^{\mathrm{TP}}$.*

Theorem 3. *If $[\![\mathcal{M}]\!]^{\mathrm{PM}}$ and $[\![\varphi]\!]^{\mathrm{PP}}$ both produce an output, then $\mathcal{M} \models_{\mathrm{S}} \varphi$ if and only if $[\![\mathcal{M}]\!]^{\mathrm{PM}} \models_{\mathrm{P}} [\![\varphi]\!]^{\mathrm{PP}}$.*

Since SAPIC$^+$'s input language is essentially equivalent to ProVerif's save for its native support for state, each ProVerif model is a valid SAPIC$^+$ model, modulo minor details. Further, every ProVerif property has a trivial translation to a SAPIC$^+$ property, since SAPIC$^+$'s property specification language is more expressive and can encode derivability and correspondence properties. Therefore, one can easily build a translation from ProVerif to Tamarin using SAPIC$^+$'s $[\![.]\!]^{\mathrm{TM}}$ and $[\![.]\!]^{\mathrm{TP}}$ functions.

These functions are also useful for bridging theoretical results between formalisms. Suppose I have a function F that simplifies applied π-calculus models (in other words, for a given π-calculus model $\mathcal{M}$, $F(\mathcal{M}) = \mathcal{M}'$, where $\mathcal{M}'$ is easier to reason about than $\mathcal{M}$). Further, suppose I have proven a theorem stating that for applied π-calculus models, F preserves validity, or lack thereof, of the properties of interest. If I want to take this result and use it for Tamarin models, I can take any applied π-calculus model $\mathcal{M}$ and compute $[\![F(\mathcal{M})]\!]^{\mathrm{TM}}$ to soundly obtain a Tamarin model with the corresponding simplification.

Could add details of the translations, examples

8.8.2 MSR to Applied π

We will outline the three major obstacles to straightforward translation from multiset rewriting models to applied π-calculus models. In particular, we are interested in practical (often-terminating) verification algorithms.

Equational theories. We already discussed the differing support for equational theories in Sections 8.2 and 8.3. In short, the termination of ProVerif's saturation algorithm relies on "compiling away" equational theories, which (to the best of my knowledge) is only well-understood for convergent or linear theories. In contrast, Tamarin's backwards search exploits the finite variant property to achieve decidable unification and with respect to a broader class of equational theories. Tamarin intentionally forgoes some automation because it has stronger support for interactivity. In principal, one could take a similar approach with ProVerif, but meaningful interactivity in their saturation-based algorithm currently seems out of reach.

State. Multiset rewriting naturally supports (linear) protocol state—a rewrite rule can reference a state fact, consume it from the global state, and produce resulting state facts. In Section **??**, we saw that ProVerif's lack of precision regarding state leads to spurious attacks. In principle, one *can* encode state precisely in the applied π-calculus using *private channels* (i.e., channels without adversarial influence). In particular, one could simulate references with processes of the form **new** $r.(\bar{r}\langle M\rangle \mid P)$, where r (a new private channel) serves as a reference to term M in P. In P, one could dereference with $r(x).\bar{r}\langle x\rangle.P'$, where x is equal to the term stored at reference r in P' (we "re-populate" the channel after reading the reference to allow multiple dereferences). Finally, in P, one could update M with N (i.e., $M := N$) with a process of the form $r(x).\bar{r}\langle N\rangle.P'$ (reading the value from the channel and updating it with a new value). However, straightforward encodings along these lines either lead to nontermination or still lead to spurious attacks with ProVerif's Horn clause encoding [5]. Some extensions to ProVerif handle state with additional heuristics

to combat these problems [24, 5], but they are often incompatible with other extensions to ProVerif (e.g., to handle exclusive or [53]).

Trace property specification. As discussed in Sections 8.2 and 8.3, Tamarin's property specification language is closer to standard first-order logic, while ProVerif's specification language is restricted to either derivability or correspondence queries. Since ProVerif's algorithm employing saturation by resolution of Horn clauses focuses on derivability of universally quantified clauses, neither negation nor existential quantification are supported.

8.9 Overview of Implemented Tools

We present an overview of implemented symbolic model cryptographic protocol verifiers (CPVs) in Table 1. We can partition these CPVs into four main groups.

The first group (in blue) denote *modern CPVs with unbounded analyses*. In particular, Tamarin and ProVerif dominate the rest in terms of modern usage [9]. Compared to the rest of the CPVs, Tamarin is surprisingly well-rounded—it imposes no bounds on analysis, has both attack soundness and proof soundness, supports a very general property specification language (while still supporting diff equivalence), supports a wide range of equational theories and global mutable protocol state, and partially supports reasoning about protocols with loops and linear integer arithmetic. No other tool comes close, save for ProVerif. While ProVerif admits spurious attacks, and has a somewhat brittle support for protocol state and equational theories, it makes up for it with its automation potential. On the other hand, while Maude-NPA's formal foundations are incredibly well-motivated, it simply has fewer features than Tamarin and ProVerif.

The second group (in green) denote *CPVs based on bounded model checking*. These CPVs can be efficient for reasoning with respect to bounded sessions, but they cannot prove security in the unbounded case. One may think these tools are complementary to tools from the first category (with unbounded analysis); however, in practice, the unbounded tools are often just as effective for reasoning about bounded sessions [9] while supporting more features.

The third group (in orange) denote *CPVs based on security by typing*. These tools work at the level of protocol *implementations* rather than protocol *designs*, and they exhibit impressive scalability. However, for efficient type checking, the type-based reasoning is overly strict and may reject secure programs. Additionally, the type checking depends on manually-annotated *secrecy labels*, which ascribe certain levels of sensitivity to terms or commands; the type checking algorithm rejects programs based on (potential) information flow violations according to these labels (e.g., a high-sensitivity value should not flow into a low-sensitivity variable).

The fourth group (in purple) denote *CPVs focused on equivalence properties*. It is much more difficult to automate reasoning with respect to equivalence properties, so these tools use bounded analyses and do not support very complicated protocols (e.g., no stateful protocols, looping protocols, or protocols requiring dedicated theory reasoning).

9 Symbolic model challenges

We discuss the major obstacles to reasoning about protocol security in the symbolic model.

9.1 Cryptographic Primitives

Handling of cryptographic primitives was discussed in Sections 7.1 and 7.2. To summarize, Tamarin models cryptographic primitives with a signature and equational theory, and considers equality of terms modulo the equational theory. On the other hand, ProVerif does not use equational theories, and rather models the semantics of cryptographic primitives with destructor rules $\mathcal{D}$, which have an easy translation to Horn clauses.

So far, this report has focused on examples of protocols that use encryption and decryption. Encryption and decryption are relatively straightforward to handle because the equational theory has nice properties: In the

Tool	Formalism	Analysis bounds	Formal guarantees (unbounded)	Trace props	Eq. props	Primitives/ equational theories	State/ Loops?	Dedicated theory reasoning
Tamarin [60]	multiset rewriting	none	AS, PS	guarded FOL	diff	convergent/ FVP, XOR, DH, BP	yes/ partial	LIA
ProVerif [16]	applied π-calculus, Horn clauses	none	PS	derivability, correspon-dence	diff	convergent/ FVP or linear, XOR*, DH*, BP*	yes*/no	variable/ constant integer addition
Maude-NPA [42]	term rewriting, strand spaces	none	AS, PS	attack patterns	diff	AC labels and confluent, terminating, coherent, SRI	no	no
CL-AtSe [75]	term rewriting	sessions	AS	attack states	none	XOR, DH, associative concatenation	no/no	no
OFMC [11]	Lazy ADTs	everything	AS	attack states	none	finite or subterm-convergent	no/no	no
SATMC [6]	SAT	everything	AS	attack states	none	encryption/ decryption	no/no	no
F7 [14]	refinement types, SMT	none	PS	secrecy, non-injective auth	none	encryption/ decryption	yes/yes	yes
DY* [13]	refinement types, SMT	none	PS	secrecy, non-injective auth	none	any	yes/yes	yes
DEEPSEC [26]	applied π-calculus	sessions	AS, PS	none	trace	subterm-convergent	no/no	no
SAT-Equiv [29]	applied π-calculus, SAT	everything	AS, PS	none	trace	encryption/ decryption	no/no	no

Table 1: Symbolic model cryptographic protocol verifiers

Formal guarantees are with respect to the *unbounded* security protocol verification problem.

AS $\mapsto$ attack soundness, AC $\mapsto$ attack completeness, PS $\mapsto$ proof soundness, PC $\mapsto$ proof completeness, LIA $\mapsto$ linear integer arithmetic, FVP $\mapsto$ finite variant property, SRI $\mapsto$ strongly right irreducible [34], XOR $\mapsto$ exclusive or, DH $\mapsto$ Diffie-Hellman exponentiation, BP $\mapsto$ bilinear pairings [61], AC $\mapsto$ associative-commutative.

Blue (first group) denotes *modern CPVs with unbounded analyses*. Green (second group) denotes *CPVs based on bounded model checking*. Orange (third group) denotes *CPVs based on security by typing*. Purple (fourth group) denotes *CPVs focused on equivalence properties*.

*ProVerif's support for these features relies on mutually incompatible extensions (not native).

Tamarin setting, the equations can be oriented from left to right to obtain a convergent and terminating rewriting system with the finite variant property (which are the standard requirements that Tamarin imposes on equational theories, discussed in Section 8.2). In the ProVerif setting, decryption can be precisely modeled with a destructor rule, and no equational theory is needed.

Some operators have more involved algebraic properties, which break both Tamarin's and ProVerif's standard encodings. An example primitive is exclusive or, which is can be modeled with the following equational theory, denoted $E_{\oplus}$:

$$x \oplus x = 0 \quad (1) \qquad (x \oplus y) \oplus z = x \oplus (y \oplus z) \quad (4)$$
$$x \oplus 0 = x \quad (2) \qquad x \oplus y = y \oplus x \quad (5)$$
$$(x \oplus x) \oplus y = y \quad (3)$$

Equation (3) appears to be redundant, but it is included for technical reasons outside the scope of this document (*AC-coherence*, [74]). First, notice that we cannot soundly phrase these equations as destructors. One

can attempt: just use Horn clauses $\mathsf{Attacker}(x \oplus x) \Rightarrow \mathsf{Attacker}(0), \mathsf{Attacker}(x \oplus 0) \Rightarrow \mathsf{Attacker}(x), \mathsf{Attacker}((x \oplus x) \oplus y) \Rightarrow \mathsf{Attacker}(y)$, and so on. But, we will miss attacks. Suppose we can derive $\mathsf{Attacker}(a \oplus (b \oplus (c \oplus 0)))$. Applying the identity law ($x \oplus 0 = 0$) on $c \oplus 0$, we should be able to derive $\mathsf{Attacker}(a \oplus (b \oplus c))$. However, this derivation relies on the equational theory being closed under congruence. But, with destructors, we can only shuffle back and forth with associativity and commutativity, as the left-hand side of the identity law's destructor does not match $\mathsf{Attacker}(a \oplus (b \oplus (c \oplus 0)))$ (or any term derivable from associativity and commutativity) "at the top level."

Why did this same limitation not apply to encryption and decryption? Say we can derive $\mathsf{Attacker}(\mathsf{s_enc}(\mathsf{s_enc}(m, k_2), k_1))$ as well as $\mathsf{Attacker}(k_2)$. Here, it is not sound to derive $\mathsf{Attacker}(\mathsf{s_enc}(m, k_1))$, as (by the semantics of perfect encryption) the adversary cannot manipulate an encrypted subterm.

To deal with exclusive or, an extension to ProVerif converts the input model into one without exclusive or, where the conversion provably preserves the validity (or lack thereof) of security properties [53].

Could give translation details

Why can we not use this equational theory in Tamarin? None of the semantics presented in Section 7.1, nor in Tamarin's constraint solving algorithm, breaks (in the sense of soundness). However, $E_\oplus$ is not convergent—for example, one can repeatedly apply rule (5). To avoid termination issues, Tamarin employs a specialized handling. This handling involves partitioning $E_\oplus$ into two equational theories, $E_{\oplus \setminus AC}$ and E_{AC}, where

$$E_{\oplus \setminus AC} = \{x \oplus x = 0, x \oplus 0 = x, (x \oplus x) \oplus y = y\} \text{ and}$$
$$E_{AC} = \{(x \oplus y) \oplus z = z \oplus (y \oplus z), x \oplus y = y \oplus x\}.$$

Now, $E_{\oplus \setminus AC}$ is convergent and has the finite variant property [38]. Then, the procedure for unification modulo E can proceed similarly to Algorithm **??**, with two updates. First, instead of computing variants under a purely convergent theory, we compute a complete set of $E_{\oplus \setminus AC}, AC$-variants of the equation $t_1 = t_2$ using a technique called folding variant narrowing modulo AC [74] (details are outside the scope of this report). Second, for each variant $(t' = s', \tau)$, we perform unification modulo AC (rather than syntactic unification) to obtain a finite set of AC-unifiers μ. For each such μ, we return the composed substitution $\tau \circ \mu$. Crucially, Algorithm **??** still terminates, as unification modulo associativity and commutativity axioms is decidable with existing algorithms from the literature [72].

Could also discuss how to make the attacker more efficient, which is a big part of the paper.

9.2 Other challenges

Existing approaches have been successful applied to analyze protocols with a variety of cryptographic primitives (e.g. exclusive or, Diffie-Hellman exponentiation, and bilinear pairings), using techniques analogous to those described in the previous subsection. However, there are many challenges where surprisingly little progress has been made. At a high level, current approaches struggle with protocols that include non-cryptographic components, non-trivial control flow (like loops), or non-trivial data structures (like recursive algebraic data types). Further, there are few techniques for compositional reasoning in the context of a single protocol, or for composing results of sub-protocols to a combined protocol. These questions are elaborated in Section 11.

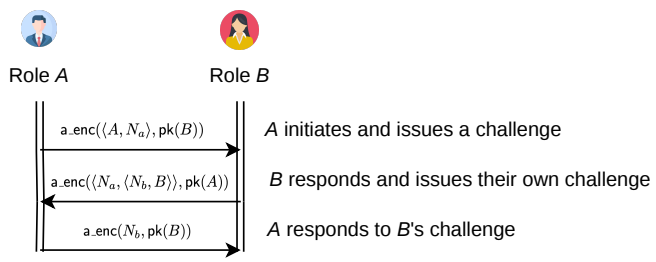
10 Tractable Reasoning in the Symbolic Model

In this section, we outline efforts to ease reasoning about security protocol verification. While symbolic model security protocol verification problem is undecidable in general [43, 49], one may recover decidability by considering a restricted class of protocols or attacks. First, we focus on typing protocols, and then we expand the discussion to a broad overview of metatheoretical results.

10.1 Type confusion attacks and the strong typing assumption

One class of attacks, called *type flaw* or *type confusion attacks*, intuitively refers to attacks involving an honest participant (mis)parsing an incoming protocol message m_1 as a different protocol message m_2. While not standard terminology, one might find it more intuitive to call them *shape confusion*, *constructor confusion*, or simply *misparsing* attacks.

First, we consider an example. Consider the following mutual authentication protocol (Figure 11), which is a variant of the Needham-Schroeder protocol in Figure 4:



Update to the original (flawed) Needham–Schroeder public-key protocol (Figure 4) by replacing $\langle N_a, N_b\rangle$ with $\langle N_a, N_b, B\rangle$.

Figure 11: Update to Needham–Schroeder public-key protocol (in Figure 4)

The only difference between this protocol and Figure 4 is that in message (2), we substituted $\langle N_a, \langle N_b, B\rangle\rangle$ for $\langle N_a, N_b\rangle$. Further, recall that we argued that using $\langle N_a, N_b, B\rangle$ (a triple, without nesting pairs) would make the protocol secure. While it is hard to believe at first glance, using the nested pairs $\langle N_a, \langle N_b, B\rangle\rangle$ rather than the triple is actually *still insecure* in the symbolic model due to a type flaw attack.

Consider the following attack trace with intruder I. In this trace, the capital letters P_1, P_2, and I denote principals (earlier, A and B denoted roles).

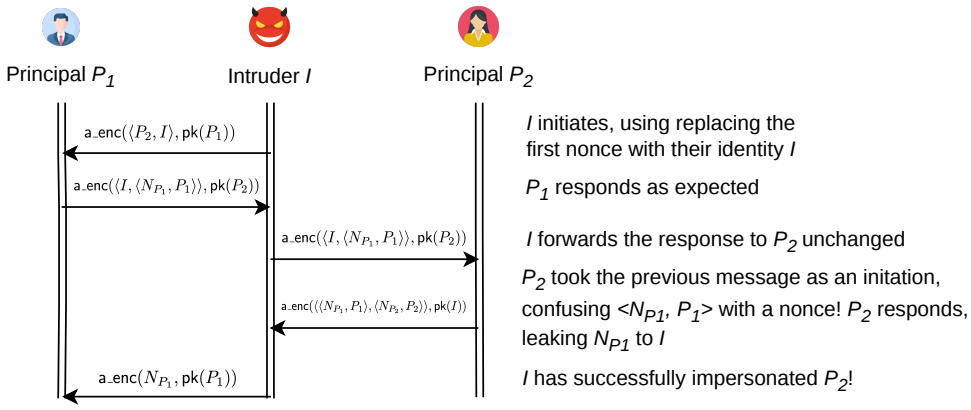


Figure 12: Type confusion attack on Updated Needham–Schroeder public-key protocol (Figure 11)

The "type confusion aspect" of the attack is exhibited in step (6) of the execution—P_2 was expecting a message of format (1) (from above), but instead got one of format (2). The attack works because P_1 does not destruct the pair in the protocol—if so, the incoming message would not unify with P_1's expected message format. This also explains why using the triple $\langle N_a, N_b, B\rangle$ is not vulnerable to the same attack.

Type confusion attacks are somewhat controversial, and for good reason. One common argument in favor of design-level verification is that if there is a design-level vulnerability, then *all* implementations will be affected. However, with type confusion attacks, this is not the case. One can imagine an implementation

of the above protocol where role B will not parse an input that substitutes a pair for a nonce, and hence the attack will not succeed. Hence, one may simply state *a priori* that only well-typed attacks will be considered—this is called the *strong typing assumption*.

10.2 Typing protocols for free

In general, the strong typing assumption is not proof sound—a proof relying on the strong typing assumption may miss an attack (one that violates the assumption). However, for a certain class of protocols, the strong typing assumption *is* proof sound—if there exists some type confusion attack, there must also exist another attack that does not rely on type confusion. In these cases, one may say we can "type the protocol for free." To understand this claim more precisely, we must first concretize the idea of typing and "well-typed attacks."

Tagging. First, we take a quick detour to note that a straightforward *tagging scheme* prevents type confusion attacks (more precisely, if a type confusion attack exists, then a well-typed attack exists). In this tagging scheme, we update each term in the protocol description $\mathsf{f}(t_1, \ldots, t_n)$ to $\mathsf{f}(\langle \mathtt{tag}_i, t_1\rangle, \ldots, t_n)$, each with a distinct $\mathtt{tag}_i$ (a concrete constant), and we update parsing to accommodate for the pair and require that honest participants check the tags to see that they match the expected values. The result is formally proven in [48]. It is not a trivial result, as the attacker may modify tags if they are not cryptographically protected.

Typing systems. A *typing system* is a pair (T, δ) such that T is a set of types and δ is a mapping from $\mathcal{T}(\Sigma_c, D)$ to T. A *well-typed substitution* σ preserves types—that is, $\delta(v) = \delta(v\sigma)$ for all $v \in V$. We call a typing system *well-formed* if (1) well-typed substitutions preserve types for every term t, i.e., $\delta(t) = \delta(t\sigma)$, and (2) if $\delta(t) = \delta(t')$, then $mgu(t, t')$ is well-typed (for all terms t, t'). A protocol trace is called *well-typed* if all the active substitutions constitute a well-typed substitution.

Example typing system. Consider an example typing system that captures term structure with a given set of atomic types $T_{\mathsf{atomic}} \subseteq T$. For example, we want the type of $\mathsf{f}(a, \mathsf{g}(b, c))$ to be $\mathsf{f}(\tau_a, \mathsf{g}(\tau_b, \tau_c))$, where the τ_is are not necessarily distinct and each τ_i represents the atomic type of data item i. We define δ as follows:

$$\delta(t) = \begin{cases} \mathsf{f}(\delta(t_1), \ldots, \delta(t_n)) & \text{if } t = \mathsf{f}(t_1, \ldots, t_n), \\ \text{some } \tau \in T_{\mathsf{atomic}} & \text{if } t \text{ is some } v \in V. \end{cases}$$

Type compliance. The *type compliance* requirement intuitively requires that all unifiable encrypted subterms in the protocol description have the same type. Informally, this requirement is similar in spirit to tagging—just as tagging makes distinct messages non-unifiable, type compliance makes encrypted terms of different types non-unifiable. To state this formally, we need the notion of *transparent* constructor symbols, which intuitively can be freely destructed and constructed by the adversary. More formally, a constructor symbol f is *transparent* if there exists a term $\mathsf{f}(t_1, \ldots, t_n) \in \mathcal{T}(\Sigma, \{\Box\})$ (denoting holes at data items) such that for any term $t \in \mathcal{T}(\Sigma, D)$ with top-level constructor symbol f, we have $\mathsf{f}(t_1, \ldots, t_n)\{t \mapsto \Box\}$ normalizes to t. By "normalizes", we refer to normalization with respect to application of the destructor rules viewed as term rewriting rules, and hence we must have destructors that correspond to a convergent rewrite system. For example, pairing is transparent, witnessed by the term $\langle \pi_1(\Box), \pi_2(\Box)\rangle$. Notice that for any pair p we insert into the holes, destructing and constructing (with π_1, π_2, and $\langle ., . \rangle$) always yields p. From transparent constructor symbols, we define the set of *encrypted subterms* of a term t as those subterms of t whose top-level constructor symbol is not transparent. A configuration $(\mathcal{P}, \phi, \sigma)$ is *type compliant* with respect to a well-formed typing system (T, δ) if, for all encrypted subterms t, t' in $\mathcal{P}$ (in channel outputs or equality tests), the image of ϕ, or the image of σ, then we have that there exists some substitution θ such that $t\theta = t'\theta$ implies $\delta(t) = \delta(t')$ (i.e., unifiable subterms have the same type). An execution $\mathcal{K} \xrightarrow{\pi:\, \mathsf{tr}} (\mathcal{P}, \phi, \sigma)$ is *well-typed* if σ is a well-typed substitution. The notation $\overline{tr}$ denotes the trace skeleton obtained from tr by replacing each $\mathsf{in}(c, R)$ with $\mathsf{in}(c, \Box)$ and each $\mathsf{out}(c, w)$ with $\mathsf{out}(c, \Box)$.

Theorem 4. *Assume $\mathcal{D}$ is a convergent rewriting system and (T, δ) is a well-formed typing system. Further, assume configuration $\mathcal{K}$ is type compliant with respect to (T, δ) and $\mathcal{D}$. If $\mathcal{K} \xrightarrow{\pi:\, \mathsf{tr}} \mathcal{K}'$ for some trace tr and configuration $\mathcal{K}'$, then there exists trace tr' such that $\mathcal{K} \xrightarrow{\pi:\, \mathsf{tr}'} \mathcal{K}'$, tr' is well-typed, and $\overline{\mathsf{tr}} = \overline{\mathsf{tr}'}$.*

10.3 Metatheorems of Symbolic Model Formalisms

The symbolic model security protocol verification problem is undecidable [43, 49]. Cutting ill-typed traces out of the search space (as discussed in Section 10.2) is one means to heuristically cope with undecidability, but there are many more—too many to cover in depth. The various attempts are summarized in Table 2.

The first group (in blue) denotes approaches that *sacrifice completeness* for decidability. These works consider a bounded variant of the verification problem (bounding sessions, message sizes, or data items), and hence they can achieve decidability on the *bounded* variant. Interestingly, while bounding sessions alone is enough to achieve decidability [70], bounding message (term) depth or the size of $\mathcal{D}$ (i.e., the number of fresh nonces) *alone* still yields undecidability [39].

The second group (in green) *syntactically restricts protocols* to achieve decidability. For example, ping-pong protocols [36] intuitively require that a message be passed back and forth ("ping-ponged"), where each subsequent message must be derived purely from cryptographic operations and the previous message without introducing extra data (e.g., fresh nonces). Then, the decidability proofs take advantage of the extra structural requirements of the protocol to soundly bound the search space.

The third group (in orange) investigates ways to *heuristically cut the search space* with no decidability guarantee (our discussion of well-typed attacks in Section 10.2 falls in this group [27]), the problem is still undecidable. Even without decidability, these results can be useful to inform the development of verification tools that perform well in practice.

The fourth group (in purple) denotes approaches that *sacrifice soundness* to achieve termination. These papers describe abstract interpretation-based approaches to over-approximate adversarial knowledge. Hence, the tools may raise spurious attacks, similarly to the "secrecy by typing" approach [14, 13].

11 Open Research Problems

In light of previous work on symbolic model verification, we now discuss some open research questions.

RQ1. *How to reason about cryptographic protocols with heavy non-cryptographic aspects in a single formalism, while also retaining some automation?* As discussed in the introduction, many protocols like 4G/5G LTE (as well as the Uptane [52] over-the-air software update protocol) involve heavy non-cryptographic reasoning. Prior work [51, 50, 55] maintains some automation, but it relies on two separate protocol models, one in a standard CPV (like ProVerif or Tamarin) and another in a general-purpose model checker (like Kind 2 or nuXmv). Ideally, reasoning could proceed with "one model to rule them all." Note that none of the tools in Table 1 have robust, dedicated theory reasoning, except for the third group (F7 and DY*). However, these tools raise spurious violations through their usage of manually-annotated secrecy labels. Additionally, one could achieve these goals with an ITP; however, this approach abandons automation.

RQ2. *How to reason about protocols with unbounded loops and recursive datatypes, without resorting to manually annotated secrecy labels (or more generally, spurious violations)?* Of the tools in Table 1, only Tamarin has *partial* support for protocols with loops (requiring expertise with non-standard modeling techniques in Tamarin [54]), and none natively support recursive datatypes, save for the third group (F7 and DY*). Again, one must either accept spurious violations (from F7 or DY*) or abandon automation (by using an ITP).

RQ3. *How to reason about cryptographic protocols compositionally?* Besides the third group (F7 and DY*), all of the tools in Table 1 reason monolithically. In the context of compositional verification, *automatically inferring* a good decomposition is a very hard problem in general, and it is often less efficient than a monolithic approach Q for Cesare—citations for this claim?. There is prior work that investigates compositional reasoning in the context of separation logic [7]; however, it suffers from the similar downsides to F7 and DY*. While automatically inferring decompositions is difficult in general, one might intuit that it could be feasible in the context of security protocols—lots of protocols have "similar" shapes with honest steps and an adversaries with similar sets of capabilities.

RQ4. *How to reason about the composition of cryptographic protocols?* Consider two protocols P_1 and P_2, and assume you have proofs that P_1 and P_2 both (independently) satisfy security property φ (i.e., $P_1 \models \varphi$ and $P_2 \models \varphi$). Then what can you say about the asynchronous parallel composition of P_1 and P_2? One cannot lift the results of the separate analyses to the composed case in any straightforward way due to adversarial influence—the adversary may use information learned from some run of P_1 to adversely influence a run of P_2.

RQ5. *How to recover useful interactivity when automated techniques fail to terminate or produce spurious violations?* A weakness of all tools in Table 1 is that when verification fails (through nontermination or spurious violations), there is no effective means of isolating the problem. Again, ITPs are strong here, but lack automation. Ideally, a tool would have high automation potential, but when the proof fails, there would be a means of interaction that isolates a hole in the proof with unfinished (local) goals.

12 Conclusion

Symbolic model cryptographic protocol verification has developed into a "killer application" for formal methods and automated reasoning, as it focuses on problems that are hard to get right but still tractable for (semi-)automated techniques. Multiset rewriting and applied π-calculus have been intensely studied to that end for many years, both in terms of theory and practical algorithms. However, there are fundamental obstacles to advancing symbolic model verification in these settings. One major obstacle is that these formalisms are inherently monolithic, which is in conflict with usable interactivity—for humans, it is easier to conceptualize decomposed problems. Further, while the existing algorithms work well for cryptographic operations, they do not generalize well for protocols heavy on non-cryptographic reasoning. In these aspects, the cryptographic protocol verification community may be able to draw inspiration from existing approaches in formal methods to address these issues.

References

[1] 3rd Generation Partnership Project. Evolved Universal Terrestrial Radio Access (E-UTRA) and Evolved Universal Terrestrial Radio Access Network (E-UTRAN); Overall Description. Technical Specification TS 36.300, 3GPP, 2022.

[2] 3rd Generation Partnership Project. NR; NR and NG-RAN Overall Description; Stage 2. Technical Specification TS 38.300, 3GPP, 2022.

[3] Martín Abadi and Cédric Fournet. Mobile values, new names, and secure communication. In *Proceedings of the 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '01, page 104–115, New York, NY, USA, 2001. Association for Computing Machinery.

[4] Martin Abadi and Phillip Rogaway. Reconciling two views of cryptography (the computational soundness of formal encryption). *Journal of cryptology*, 15(2):103–127, 2002.

[5] Myrto Arapinis, Joshua Phillips, Eike Ritter, and Mark D. Ryan. Statverif: Verification of stateful processes. *Journal of Computer Security*, 22(5):743–821, 2014.

[6] Alessandro Armando and Luca Compagna. Satmc: A sat-based model checker for security protocols. In Jóse Júlio Alferes and João Leite, editors, *Logics in Artificial Intelligence*, pages 730–733, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.

[7] Linard Arquint, Malte Schwerhoff, Vaibhav Mehta, and Peter Müller. A generic methodology for the modular verification of security protocol implementations. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, CCS '23, page 1377–1391, New York, NY, USA, 2023. Association for Computing Machinery.

[8] Gergei Bana and Hubert Comon-Lundh. Towards unconditional soundness: Computationally complete symbolic attacker. In Pierpaolo Degano and Joshua D. Guttman, editors, *Principles of Security and Trust*, pages 189–208, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.

[9] Manuel Barbosa, Gilles Barthe, Karthik Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao, and Bryan Parno. Sok: Computer-aided cryptography. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 777–795, 2021.

[10] Gilles Barthe, François Dupressoir, Benjamin Grégoire, César Kunz, Benedikt Schmidt, and Pierre-Yves Strub. *EasyCrypt: A Tutorial*, pages 146–166. Springer International Publishing, Cham, 2014.

[11] David Basin, Sebastian Mödersheim, and Luca Vigano. Ofmc: A symbolic model checker for security protocols. *International Journal of Information Security*, 4(3):181–208, 2005.

[12] David A. Basin, Andreas Lochbihler, and S. Reza Sefidgar. CryptHOL: Game-based proofs in higher-order logic. Cryptology ePrint Archive, Paper 2017/753, 2017.

[13] Karthikeyan Bhargavan, Abhishek Bichhawat, Quoc Huy Do, Pedram Hosseyni, Ralf Küsters, Guido Schmitz, and Tim Würtele. DY* : A Modular Symbolic Verification Framework for Executable Cryptographic Protocol Code. In *EuroS&P 2021 - 6th IEEE European Symposium on Security and Privacy*, Virtual, Austria, September 2021.

[14] Karthikeyan Bhargavan, Cédric Fournet, and Andrew D. Gordon. Modular verification of security protocol code by typing. In *Proceedings of the 37th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '10, page 445–456, New York, NY, USA, 2010. Association for Computing Machinery.

[15] Bruno Blanchet. Automatic verification of correspondences for security protocols*. *Journal of Computer Security*, 17(4):363–434, 2009.

[16] Bruno Blanchet. *Automatic Verification of Security Protocols in the Symbolic Model: The Verifier ProVerif*, pages 54–87. Springer International Publishing, Cham, 2014.

[17] Bruno Blanchet, Martín Abadi, and Cédric Fournet. Automated verification of selected equivalences for security protocols. *The Journal of Logic and Algebraic Programming*, 75(1):3–51, 2008. Algebraic Process Calculi. The First Twenty Five Years and Beyond. III.

[18] Bruno Blanchet and Andreas Podelski. Verification of cryptographic protocols: Tagging enforces termination. In Andrew D. Gordon, editor, *Foundations of Software Science and Computation Structures*, pages 136–152, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.

[19] Bruno Blanchet, Ben Smyth, Vincent Cheval, and Marc Sylvestre. Proverif 2.00: automatic cryptographic protocol verifier, user manual and tutorial. *Version from*, 16:05–16, 2018.

[20] Daniel Bleichenbacher. Chosen ciphertext attacks against protocols based on the rsa encryption standard pkcs #1. In Hugo Krawczyk, editor, *Advances in Cryptology — CRYPTO '98*, pages 1–12, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg.

[21] Roberto Cavada, Alessandro Cimatti, Michele Dorigatti, Alberto Griggio, Alessandro Mariotti, Andrea Micheli, Sergio Mover, Marco Roveri, and Stefano Tonetta. The nuxmv symbolic model checker. In Armin Biere and Roderick Bloem, editors, *Computer Aided Verification*, pages 334–342, Cham, 2014. Springer International Publishing.

[22] I. Cervesato, N. Durgin, J. Mitchell, P. Lincoln, and A. Scedrov. Relating strands and multiset rewriting for security protocol analysis. In *Proceedings 13th IEEE Computer Security Foundations Workshop. CSFW-13*, pages 35–51, 2000.

[23] Adrien Champion, Alain Mebsout, Christoph Sticksel, and Cesare Tinelli. The kind 2 model checker. In Swarat Chaudhuri and Azadeh Farzan, editors, *Computer Aided Verification*, pages 510–517, Cham, 2016. Springer International Publishing.

[24] Vincent Cheval, Véronique Cortier, and Mathieu Turuani. A little more conversation, a little less action, a lot more satisfaction: Global states in proverif. In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*, pages 344–358, 2018.

[25] Vincent Cheval, Charlie Jacomme, Steve Kremer, and Robert Künnemann. SAPIC+: protocol verifiers of the world, unite! In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3935–3952, Boston, MA, August 2022. USENIX Association.

[26] Vincent Cheval, Steve Kremer, and Itsaka Rakotonirina. Deepsec: Deciding equivalence properties in security protocols theory and practice. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 529–546, 2018.

[27] Rémy Chrétien, Véronique Cortier, Antoine Dallon, and Stéphanie Delaune. Typing messages for free in security protocols. *ACM Trans. Comput. Logic*, 21(1), September 2019.

[28] Michael R. Clarkson and Fred B. Schneider. Hyperproperties. *Journal of Computer Security*, 18(6):1157–1210, 2010.

[29] Véronique Cortier, Antoine Dallon, and Stéphanie Delaune. Sat-equiv: An efficient tool for equivalence properties. In *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*, pages 481–494, 2017.

[30] Véronique Cortier, Antoine Dallon, and Stéphanie Delaune. A small bound on the number of sessions for security protocols. In *2022 IEEE 35th Computer Security Foundations Symposium (CSF)*, pages 33–48, 2022.

[31] Véronique Cortier, Stéphanie Delaune, Jannik Dreier, and Elise Klein. Automatic generation of sources lemmas in tamarin: Towards automatic proofs of security protocols1. *Journal of Computer Security*, 30(4):573–598, 2022.

[32] Stéphanie Delaune and Lucca Hirschi. A survey of symbolic methods for establishing equivalence-based properties in cryptographic protocols. *Journal of Logical and Algebraic Methods in Programming*, 87:127–144, 2017.

[33] Stéphanie Delaune, Steve Kremer, and Mark Ryan. Verifying privacy-type properties of electronic voting protocols. *Journal of Computer Security*, 17(4):435–487, 2009.

[34] Nachum DERSHOWITZ and Jean-Pierre JOUANNAUD. Chapter 6 - rewrite systems. In JAN VAN LEEUWEN, editor, *Formal Models and Semantics*, Handbook of Theoretical Computer Science, pages 243–320. Elsevier, Amsterdam, 1990.

[35] Whitfield Diffie, Paul C Van Oorschot, and Michael J Wiener. Authentication and authenticated key exchanges. *Designs, Codes and cryptography*, 2(2):107–125, 1992.

[36] D. Dolev, S. Even, and R.M. Karp. On the security of ping-pong protocols. *Information and Control*, 55(1):57–68, 1982.

[37] D. Dolev and A. Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983.

[38] Jannik Dreier, Lucca Hirschi, Sasa Radomirovic, and Ralf Sasse. Automated unbounded verification of stateful cryptographic protocols with exclusive or. In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*, pages 359–373, 2018.

[39] Nancy Durgin, Patrick Lincoln, John Mitchell, and Andre Scedrov. Multiset rewriting and the complexity of bounded security protocols. *Journal of Computer Security*, 12(2):247–311, 2004.

[40] Nancy A Durgin and DC Mitchell. Analysis of security protocols. *NATO ASI SERIES F COMPUTER AND SYSTEMS SCIENCES*, 173:369–394, 1999.

[41] Emanuele D'Osualdo, Luke Ong, and Alwen Tiu. Deciding secrecy of security protocols for an unbounded number of sessions: The case of depth-bounded processes. In *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*, pages 464–480, 2017.

[42] Santiago Escobar, Catherine Meadows, and José Meseguer. *Maude-NPA: Cryptographic Protocol Analysis Modulo Equational Properties*, pages 1–50. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009.

[43] S. Even and O. Goldreich. On the security of multi-party ping-pong protocols. In *24th Annual Symposium on Foundations of Computer Science (sfcs 1983)*, pages 34–39, 1983.

[44] F.J.T. Fabrega, J.C. Herzog, and J.D. Guttman. Strand spaces: why is a security protocol correct? In *Proceedings. 1998 IEEE Symposium on Security and Privacy (Cat. No.98CB36186)*, pages 160–171, 1998.

[45] Oded Goldreich. *Foundations of cryptography, volume 2*. Cambridge university press Cambridge, 2004.

[46] Jean Goubault-Larrecq. A method for automatic cryptographic protocol verification. In José Rolim, editor, *Parallel and Distributed Processing*, pages 977–984, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.

[47] J. Heather and S. Schneider. Towards automatic verification of authentication protocols on an unbounded network. In *Proceedings 13th IEEE Computer Security Foundations Workshop. CSFW-13*, pages 132–143, 2000.

[48] James Heather, Gavin Lowe, and Steve Schneider. How to prevent type flaw attacks on security protocols. *Journal of Computer Security*, 11(2):217–244, 2003.

[49] N. Heintze and J.D. Tygar. A model for secure protocols and their compositions. *IEEE Transactions on Software Engineering*, 22(1):16–30, 1996.

[50] S. Hussain, O. Chowdhury, S. Mehnaz, and E. Bertino. Lteinspector: A systematic approach for adversarial testing of 4g lte. *Network and Distributed Systems Security (NDSS) Symposium 2018*.

[51] Syed Rafiul Hussain, Mitziu Echeverria, Imtiaz Karim, Omar Chowdhury, and Elisa Bertino. 5greasoner: A property-directed security and privacy analysis framework for 5g cellular network protocol. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, CCS '19, page 669–684, New York, NY, USA, 2019. Association for Computing Machinery.

[52] Trishank Karthik, Akan Brown, Sebastien Awwad, Damon McCoy, Russ Bielawski, Cameron Mott, Sam Lauzon, André Weimerskirch, and Justin Cappos. Uptane: Securing software updates for automobiles. In *International conference on embedded security in car*, volume 11, 2016.

[53] Ralf Küsters and Tomasz Truderung. Reducing protocol analysis with xor to the xor-free case in the horn theory based approach. In *Proceedings of the 15th ACM Conference on Computer and Communications Security*, CCS '08, page 129–138, New York, NY, USA, 2008. Association for Computing Machinery.

[54] Felix Linker, Christoph Sprenger, Cas Cremers, and David Basin. Looping for good: Cyclic proofs for security protocols. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS '25, page 2759–2773, New York, NY, USA, 2025. Association for Computing Machinery.

[55] Robert Lorch, Daniel Larraz, Cesare Tinelli, and Omar Chowdhury. A comprehensive, automated security analysis of the uptane automotive over-the-air update framework. In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses*, RAID '24, page 594–612, New York, NY, USA, 2024. Association for Computing Machinery.

[56] G. Lowe. A hierarchy of authentication specifications. In *Proceedings 10th Computer Security Foundations Workshop*, pages 31–43, 1997.

[57] G. Lowe. Towards a completeness result for model checking of security protocols. In *Proceedings. 11th IEEE Computer Security Foundations Workshop (Cat. No.98TB100238)*, pages 96–105, 1998.

[58] Gavin Lowe. An attack on the needham- schroeder public- key authentication protocol. *Information processing letters*, 56(3), 1995.

[59] Alberto Martelli and Ugo Montanari. An efficient unification algorithm. *ACM Trans. Program. Lang. Syst.*, 4(2):258–282, April 1982.

[60] Simon Meier. *Advancing automated security protocol verification.* PhD thesis, ETH, 2013.

[61] Alfred Menezes. An introduction to pairing-based cryptography. *Recent trends in cryptography*, 477:47–65, 2009.

[62] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28*, pages 625–635, Cham, 2021. Springer International Publishing.

[63] Roger M. Needham and Michael D. Schroeder. Using encryption for authentication in large networks of computers. *Commun. ACM*, 21(12):993–999, December 1978.

[64] Flemming Nielson, Hanne Riis Nielson, and Helmut Seidl. Cryptographic analysis in cubic time. *Electronic Notes in Theoretical Computer Science*, 62:7–23, 2002. TOSCA 2001, Theory of Concurrency, Higher Order Languages and Types.

[65] Lawrence C Paulson. *Isabelle: A generic theorem prover.* Springer, 1994.

[66] Lawrence C. Paulson. The inductive approach to verifying cryptographic protocols. *Journal of Computer Security*, 6(1-2):85–128, 1998.

[67] Strategic Planning. The economic impacts of inadequate infrastructure for software testing. *National Institute of Standards and Technology*, 1(2002):1, 2002.

[68] R. Ramanujam and S. P. Suresh. Tagging makes secrecy decidable with unbounded nonces as well. In Paritosh K. Pandya and Jaikumar Radhakrishnan, editors, *FST TCS 2003: Foundations of Software Technology and Theoretical Computer Science*, pages 363–374, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.

[69] R. Ramanujam and S.P. Suresh. Decidability of context-explicit security protocols. *Journal of Computer Security*, 13(1):135–165, 2005.

[70] Michaël Rusinowitch and Mathieu Turuani. Protocol Insecurity with Finite Number of Sessions is NP-complete. Research Report RR-4134, INRIA, 2001.

[71] William Stallings. *Cryptography and Network Security: Principles and Practice.* Pearson Education, 3rd edition, 2002.

[72] Mark E Stickel. A unification algorithm for associative-commutative functions. *Journal of the ACM (JACM)*, 28(3):423–434, 1981.

[73] Alwen Tiu. A trace based bisimulation for the spi calculus: An extended abstract. In Zhong Shao, editor, *Programming Languages and Systems*, pages 367–382, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.

[74] Mathieu Turuani. The cl-atse protocol analyser. In Frank Pfenning, editor, *Term Rewriting and Applications*, pages 277–286, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.

[75] Mathieu Turuani. The cl-atse protocol analyser. In Frank Pfenning, editor, *Term Rewriting and Applications*, pages 277–286, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.

[76] M. H. Van Emden and R. A. Kowalski. The semantics of predicate logic as a programming language. *J. ACM*, 23(4):733–742, October 1976.

13 Appendix

14 Notes

- Make sure to give examples for abstract concepts

- Give tables and figures/visualizations where possible
- Could also discuss different notions of equivalence, and some are discussed in the "Typing messages for free" paper. And maybe should give a deeper discussion of reachability vs equivalence somewhere in the intro. Observational equivalence vs trace equivalence, and intuition. There are other notions (diff equivalence, may-testing, variants of bisimulation, etc.) but maybe shouldn't go into details.
- Table of CPVs, what they support, reasoning engines, etc.

15 Questions

- In metatheorems, security problem vs secrecy problem?
- How can there be distinct MGUs?
- What part of unification algo + AC breaks if we don't have AC-coherence, AC-convergence?
- How is proverif's overapproximation of attacker-computable terms different from secrecy by typing world, e.g. DY*? A: Overapprox has to do with protocol control flow, which is somewhat orthogonal to overapprox due to secrecy labels and lack of "smallest relation containing" semantics (which ProVerif gets from resolution algo)
- What are the 0-ary constants in the Herbrand universe of ProVerif's Horn clause representation? It seems like in the Horn clause translation, "patterns" produce function symbols (including 0-arity)
- Why does the typing protocols for free paper require convergent destructor rules? Why can't we work with unnormalized terms? One part of it is that the definition of "transparent" relies on normal forms, but this seems like it could be worked around.
- Where are the structural equivalence rules in "typing protocols for free"? A: Captured in "Par" and semantics of multisets

Paper	**Formalism**	**Restrictions**	**Benefit**	**Fragment decidable?**	**Formal guarantees** (wrt unbounded)
Rusinowitch & Turuani [70]	DY	bounded sessions	NP-completeness	yes	AS
Durgin eg al. [39]	multiset rewriting	bound message size and $\lvert\mathcal{D}\rvert$	EXPTIME-completeness	yes	AS
Ramanujam & Suresh [68]	DY	tagged protocols, bounded messages, secrecy properties	decidability	yes	AC, PC
Dolev et al. [36]	DY	ping-pong [36] protocols	PTIME-completeness	yes	AS, PS, AC, PC
Ramanujam & Suresh [69]	DY	context-explicit [69] protocols, secrecy properties	NEXP-time	yes	AS, PS, AC, PC
D'Osualdo et al. [41]	applied π-calculus	depth-bounded protocols [41]	decidability	yes	AS, PS, AC, PC
Lowe [57]	DY	context-explicit, inferrable identities, no temporary secrets, secrecy properties	small model theorem (one session with one principal per role)	yes	AS, PS, AC, PC
Cortier et al. [30]	applied π-calculus	Type compliance, simple protocols*, no else branches*	heuristically soundly bound number of sessions	no	AS, PS
Chrétien et al. [27]	applied π-calculus	type compliance, convergent equational theory	cut ill-typed attacks from search space	no	PS, AS
Lowe & Schneider [48]	strand spaces	tagged protocols	cut ill-typed attacks from search space	no	PS, AS
Blanchet & Podelski [18]	applied π-calculus/ Horn clauses	tagged protocols	termination	no	PS
Nielson et al. [64]	DY	secrecy properties	termination (through approximation)	yes	PS, AC
Goubault-Larrecq [46]	DY	secrecy properties	termination (through approximation)	yes	PS, AC
Heather & Schneider [47]	CSP	auth properties	termination (through approximation)	yes	PS, AC

DY is shorthand for any formalism that closely follows the original Dolev-Yao encoding [37]. It is very similar in spirit to strand spaces [44], and can only encode simple (stateless, purely cryptographic) protocols.

Blue (first group) denotes approaches *sacrificing completeness for decidability*. Green (second group) denotes approaches *syntactically restricting protocols for decidability*. Orange (third group) denotes approaches to *heuristically cut the search space* (without a termination guarantee). Purple (fourth group) denotes approaches *sacrificing soundness for termination*.

*Only needed for equivalence properties.

Table 2: Symbolic model metatheorems

$P_0 = \emptyset,\ L_0 = \emptyset.$

$\{\} \xrightarrow{\{\}} \{\mathsf{Fr}(sk_A)\}$
$P_1 = P_0, \qquad L_1 = \{\mathsf{Fr}(sk_A)\}.$

$\{\mathsf{Fr}(sk_A)\} \xrightarrow{\{\}} \{\mathsf{Ltk}(A, sk_A), \mathsf{Pk}(A, \mathsf{pk}(sk_A)), \mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle)\}$
$P_2 = P_1 \cup^{\#} \{\mathsf{Ltk}(A, sk_A), \mathsf{Pk}(A, \mathsf{pk}(sk_A))\},$
$L_2 = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle)\}.$

$\{\} \xrightarrow{\{\}} \{\mathsf{Fr}(sk_I)\}$
$P_3 = P_2,$
$L_3 = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Fr}(sk_I)\}.$

$\{\mathsf{Fr}(sk_I)\} \xrightarrow{\{\}} \{\mathsf{Ltk}(I, sk_I), \mathsf{Pk}(I, \mathsf{pk}(sk_I)), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle)\}$
$P_4 = P_3 \cup^{\#} \{\mathsf{Ltk}(I, sk_I), \mathsf{Pk}(I, \mathsf{pk}(sk_I))\},$
$L_4 = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle)\}.$

$\{\} \xrightarrow{\{\}} \{\mathsf{Fr}(sk_B)\}$
$P_5 = P_4,$
$L_5 = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle), \mathsf{Fr}(sk_B)\}.$

$\{\mathsf{Fr}(sk_B)\} \xrightarrow{\{\}} \{\mathsf{Ltk}(B, sk_B), \mathsf{Pk}(B, \mathsf{pk}(sk_B)), \mathsf{Out}(\langle B, \mathsf{pk}(sk_B)\rangle)\}$
$P_6 = P_5 \cup^{\#} \{\mathsf{Ltk}(B, sk_B), \mathsf{Pk}(B, \mathsf{pk}(sk_B))\},$
$L_6 = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle), \mathsf{Out}(\langle B, \mathsf{pk}(sk_B)\rangle)\}.$

$\{\} \xrightarrow{\{\}} \{\mathsf{Fr}(n_A)\}$
$P_7 = P_6,$
$L_7 = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle), \mathsf{Out}(\langle B, \mathsf{pk}(sk_B)\rangle), \mathsf{Fr}(n_A)\}.$

$\{\mathsf{Fr}(n_A)\} \xrightarrow{\{\}} \{\mathsf{StA1}(A, I, n_A), \mathsf{Out}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_I)))\}$
$P_8 = P_7,$
$L_8 = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle), \mathsf{Out}(\langle B, \mathsf{pk}(sk_B)\rangle), \mathsf{StA1}(A, I, n_A), \mathsf{Out}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_I)))\}.$

$\{\mathsf{Out}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_I)))\} \xrightarrow{\{\}} \{\mathsf{K}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_I)))\}$
$P_9 = P_8,$
$L_9 = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle), \mathsf{Out}(\langle B, \mathsf{pk}(sk_B)\rangle), \mathsf{StA1}(A, I, n_A),$
$\quad \mathsf{K}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_I)))\}.$

$\{\mathsf{K}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_B)))\} \xrightarrow{\{\mathsf{K}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_B)))\}} \{\mathsf{In}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_B)))\}$
$P_{10} = P_9,$
$L_{10} = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle), \mathsf{Out}(\langle B, \mathsf{pk}(sk_B)\rangle), \mathsf{StA1}(A, I, n_A),$
$\quad \mathsf{K}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_I))), \mathsf{In}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_B)))\}.$

P_i (resp., L_i) denotes the persistent (resp., linear) state at step i

Figure 13: Needham–Schroeder public-key protocol attack as multiset rewriting protocol trace

$\{\} \xrightarrow{\{\}} \{\mathsf{Fr}(n_B)\}$
$P_{11} = P_{10},$
$L_{11} = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle), \mathsf{Out}(\langle B, \mathsf{pk}(sk_B)\rangle), \mathsf{StA1}(A, I, n_A),$
$\mathsf{K}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_I))), \mathsf{In}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_B))), \mathsf{Fr}(n_B)\}.$

$\{\mathsf{In}(c), \mathsf{Fr}(n_B)\} \xrightarrow{\{\}} \{\mathsf{StB1}(B, A, n_A, n_B), \mathsf{Out}(\mathsf{a_enc}(\langle n_A, n_B\rangle, \mathsf{pk}(sk_A)))\}$
$(c =_E \mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_B)))$
$P_{12} = P_{11},$
$L_{12} = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle), \mathsf{Out}(\langle B, \mathsf{pk}(sk_B)\rangle), \mathsf{StA1}(A, I, n_A), \mathsf{K}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_I))),$
$\mathsf{StB1}(B, A, n_A, n_B), \mathsf{Out}(\mathsf{a_enc}(\langle n_A, n_B\rangle, \mathsf{pk}(sk_A)))\}.$

$\{\mathsf{Out}(m)\} \xrightarrow{\{\}} \{\mathsf{K}(m)\} \quad (m = \mathsf{a_enc}(\langle n_A, n_B\rangle, \mathsf{pk}(sk_A)))$
$P_{13} = P_{12},$
$L_{13} = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle), \mathsf{Out}(\langle B, \mathsf{pk}(sk_B)\rangle), \mathsf{StA1}(A, I, n_A), \mathsf{K}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_I))),$
$\mathsf{StB1}(B, A, n_A, n_B), \mathsf{K}(\mathsf{a_enc}(\langle n_A, n_B\rangle, \mathsf{pk}(sk_A)))\}.$

$\{\mathsf{K}(x)\} \xrightarrow{\{\mathsf{K}(x)\}} \{\mathsf{In}(x)\} \quad (x = \mathsf{a_enc}(\langle n_A, n_B\rangle, \mathsf{pk}(sk_A)))$
$P_{14} = P_{13},$
$L_{14} = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle), \mathsf{Out}(\langle B, \mathsf{pk}(sk_B)\rangle), \mathsf{StA1}(A, I, n_A),$
$\mathsf{K}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_I))), \mathsf{StB1}(B, A, n_A, n_B), \mathsf{In}(\mathsf{a_enc}(\langle n_A, n_B\rangle, \mathsf{pk}(sk_A)))\}.$

$\{\mathsf{In}(c), \mathsf{StA1}(A, I, n_A)\} \xrightarrow{\{\}} \{\mathsf{StA2}(A, I, n_A, n_B), \mathsf{Out}(\mathsf{a_enc}(n_B, \mathsf{pk}(sk_I)))\}$
$(c =_E \mathsf{a_enc}(\langle n_A, n_B\rangle, \mathsf{pk}(sk_A)))$
$P_{15} = P_{14},$
$L_{15} = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle), \mathsf{Out}(\langle B, \mathsf{pk}(sk_B)\rangle), \mathsf{K}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_I))), \mathsf{StB1}(B, A, n_A, n_B),$
$\mathsf{StA2}(A, I, n_A, n_B), \mathsf{Out}(\mathsf{a_enc}(n_B, \mathsf{pk}(sk_I)))\}.$

$\{\mathsf{Out}(m)\} \xrightarrow{\{\}} \{\mathsf{K}(m)\} \quad (m = \mathsf{a_enc}(n_B, \mathsf{pk}(sk_I)))$
$P_{16} = P_{15},$
$L_{16} = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle), \mathsf{Out}(\langle B, \mathsf{pk}(sk_B)\rangle), \mathsf{K}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_I))), \mathsf{StB1}(B, A, n_A, n_B),$
$\mathsf{StA2}(A, I, n_A, n_B), \mathsf{K}(\mathsf{a_enc}(n_B, \mathsf{pk}(sk_I)))\}.$

$\{\mathsf{K}(x)\} \xrightarrow{\{\mathsf{K}(x)\}} \{\mathsf{In}(x)\} \quad (x = \mathsf{a_enc}(n_B, \mathsf{pk}(sk_B)))$
$P_{17} = P_{16},$
$L_{17} = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle), \mathsf{Out}(\langle B, \mathsf{pk}(sk_B)\rangle), \mathsf{K}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_I))), \mathsf{StB1}(B, A, n_A, n_B),$
$\mathsf{StA2}(A, I, n_A, n_B), \mathsf{K}(\mathsf{a_enc}(n_B, \mathsf{pk}(sk_I))), \mathsf{In}(\mathsf{a_enc}(n_B, \mathsf{pk}(sk_B)))\}.$

$\{\mathsf{In}(c)\} \xrightarrow{\{\}} \{\mathsf{Commit}(B, A, n_A, n_B)\} \quad (c =_E \mathsf{a_enc}(n_B, \mathsf{pk}(sk_B)))$
$P_{18} = P_{17},$
$L_{18} = \{\mathsf{Out}(\langle A, \mathsf{pk}(sk_A)\rangle), \mathsf{Out}(\langle I, \mathsf{pk}(sk_I)\rangle), \mathsf{Out}(\langle B, \mathsf{pk}(sk_B)\rangle), \mathsf{K}(\mathsf{a_enc}(\langle A, n_A\rangle, \mathsf{pk}(sk_I))), \mathsf{StB1}(B, A, n_A, n_B),$
$\mathsf{StA2}(A, I, n_A, n_B), \mathsf{K}(\mathsf{a_enc}(n_B, \mathsf{pk}(sk_I))), \mathsf{Commit}(B, A, n_A, n_B)\}.$

P_i (resp., L_i) denotes the persistent (resp., linear) state at step i

Figure 14: Needham–Schroeder public-key protocol attack as multiset rewriting protocol trace, part 2